



សាកលវិទ្យាល័យប្រជាជនកម្ពុជា

មហាវិទ្យាល័យវិទ្យាសាស្ត្រ ដេប៉ាតឺម៉ង់ព័ត៌មានវិទ្យា

ប្រព័ន្ធគ្រប់គ្រងការដឹកជញ្ជូនអ្នកដំណើរនិងការផ្ញើឥវ៉ាន់

(Passenger and Goods Transportation Management System)

ដឹកនាំដោយសាស្ត្រាចារ្យ៖ **ផេង ឧសភា**

មុខវិជ្ជា៖ Information System Analysis & Design



Assignment Group 5

1. ឈាន សិលាពីន (Group Leader)
2. ហួត វ៉ាលី (Group Sub-leader)
3. កែវ វិរៈ
4. កែវ ស៊ីវជានឆាត
5. គាង កៅលីឡេង
6. រ៉េត វីហ្សា
7. ពក ឈូ
8. គៀ សោវណ្ណ
9. វិន ឡាត

Content AT A GLANCE

ដំណាក់កាលទី១ ការធ្វើផែនការប្រព័ន្ធ (System Planning)

I. មុខងារនៃប្រព័ន្ធពាណិជ្ជកម្ម	១
II. ដំណើរការនៃប្រព័ន្ធពាណិជ្ជកម្ម.....	១
III. បញ្ហាកើតឡើងធ្វើឱ្យមានភាពរាំងស្ទះដល់ដំណើរការពាណិជ្ជកម្ម	៣
IV. ដំណោះស្រាយតាមបែបកុំព្យូទ័រ	៣
V. ស្ថាបត្យកម្មនៃប្រព័ន្ធព័ត៌មាន	៤
VI. ការកំណត់ពេលវេលាដែលត្រូវចំណាយលើព្រំដែនការងារ.....	៥
VII. ការកំណត់ថវិការដែលត្រូវចំណាយលើព្រំដែនការងារ	៥

ដំណាក់កាលទី២ ការវិភាគប្រព័ន្ធ (System Analysis)

I. ការធ្វើ Process Modeling.....	៦
II. ការធ្វើ Data Modeling	១០

ដំណាក់កាលទី៣ ការរចនាប្រព័ន្ធ (System Design)

I. ការបង្កើត Data Dictionary	១៨
II. ការរចនា User Interface Layout	២២

ដំណាក់កាលទី៤ ការតំឡើងប្រព័ន្ធ (System Implementation)

I. ការដំឡើងនៅផ្នែក Server(Server-Side Implementation).....	៣០
II. ការដំឡើងនៅផ្នែក Client(Client-side Implementation)	៣០
III. ការបង្កើតឯកសារសម្រាប់បង្រៀនអ្នកប្រើប្រាស់	៣១

EXPANDED CONTENTS

ដំណាក់កាលទី១ ការធ្វើផែនការប្រព័ន្ធ (System Planning)

I. មុខងារនៃប្រព័ន្ធពាណិជ្ជកម្ម	១
II. ដំណើរការនៃប្រព័ន្ធពាណិជ្ជកម្ម.....	១
III. បញ្ហាកើតឡើងធ្វើឱ្យមានភាពរាំងស្ងះដល់ដំណើរការពាណិជ្ជកម្ម	៣
IV. ដំណោះស្រាយតាមបែបកុំព្យូទ័រ	៣
V. ស្ថាបត្យកម្មនៃប្រព័ន្ធព័ត៌មាន	៤
1. តម្រូវការផ្នែករឹង (Hardware Requirements)	៤
2. តម្រូវការផ្នែកទន់ចំពោះ Server (Server Software Requirements)	៤
VI. ការកំណត់ពេលវេលាដែលត្រូវចំណាយលើព្រំដែនការងារ.....	៥
VII. ការកំណត់ថវិការដែលត្រូវចំណាយលើព្រំដែនការងារ	៥
1. ចំណាយលើផ្នែករឹង	៥
2. ចំណាយលើផ្នែកទន់	៥

ដំណាក់កាលទី២ ការវិភាគប្រព័ន្ធ (System Analysis)

I. ការធ្វើ Process Modeling.....	៦
1. ដ្យាក្រាមលំហូរទិន្នន័យ	៦
2. ប្រភេទនៃដ្យាក្រាមលំហូរទិន្នន័យ	៧
2.1 Context Diagram.....	៧
2.2 Physical Level Diagram	៨
II. ការធ្វើ Data Modeling	១០
1. ការកំណត់ Entity Sets	១០
2. ការកំណត់ Strong និង Weak Entity Sets	១០
3. ការកំណត់ Attribute និង Primary Key របស់ Entity Sets	១១

4. ការកំណត់ Relationship Entity Sets.....	១៣
5. ការគូស Entity-Relationship Diagram(ER-D)	១៦
6. ការកំណត់Table	១៦
7. ការធ្វើ De-normalization	១៧

ជំណាក់កាលទី៣ **ការរចនាប្រព័ន្ធ** (System Design)

I. ការបង្កើត Data Dictionary

1. tbCustomer	១៨
2. tbStaff.....	១៨
3. tbRoute	១៨
4. tbVehicle	១៩
5. tbItem	១៩
6. tbBooking.....	១៩
7. tbPayment.....	២០
8. tbTransport.....	២០
9. tbInvoice	២០
10. tbTicket.....	២១
11. tbUser.....	២១

II. ការរចនា User Interface Layout

1. Customer's Form	២២
2. Staff's Form	២៣
3. Route's Form	២៤
4. Vehicle's Form.....	២៤
5. Item's Form.....	២៥

6. Booking's Form	២៦
7. Payment's Form	២៧
8. Transport's Form	២៧
9. Invoice's Form.....	២៨
10. Ticket's Form	២៨
11. User's	២៩

ជំណាក់កាលទី៤ **ការងារប្រព័ន្ធ** (System Implementation)

I. ការដំឡើងនៅផ្នែក Server(Server-Side Implementation)	៣០
II. ការដំឡើងនៅផ្នែក Client(Client-side Implementation)	៣០
III. ការបង្កើតឯកសារសម្រាប់បង្រៀនអ្នកប្រើប្រាស់Form	៣១

PREFACE

សព្វថ្ងៃនេះ ព័ត៌មានវិទ្យានៅលើពិភពលោកមានការរីកចម្រើនយ៉ាងខ្លាំង និងត្រូវបានយកទៅប្រើប្រាស់ក្នុងវិស័យជាច្រើនក្នុងការអភិវឌ្ឍន៍ប្រទេសជាតិមួយ។ វាបានជួយក្នុងការដោះស្រាយបញ្ហា និងសម្រួលដល់ការងារប្រចាំថ្ងៃរបស់មនុស្សនៅលើពិភពលោកស្ទើរតែទាំងស្រុងដោយសារវាអាចដោះស្រាយបញ្ហានានាយ៉ាងមានប្រសិទ្ធភាពនិងលឿនបំផុត។

យើងខ្ញុំជានិស្សិតនៃសាកលវិទ្យាល័យភូមិន្ទភ្នំពេញ ឆ្នាំទី ៣ ខាងផ្នែកព័ត៌មានវិទ្យាបង្រៀនដោយលោកគ្រូ ផេង ឧសភា នូវមុខវិជ្ជា ការវិភាគនិងរចនាប្រព័ន្ធព័ត៌មាន ដែលគាត់បានអោយយើងសិក្សាស្រាវជ្រាវ និងប្រើប្រាស់ System Development Life Cycle (SDLC) ដើម្បីបង្កើតនូវប្រព័ន្ធគ្រប់គ្រងការដឹកជញ្ជូនអ្នកដំណើរនិងការផ្ញើត្រីវ៉ាន់ ។ គោលបំណងនៃការស្រាវជ្រាវនេះគឺគាត់ចង់អោយយើងខ្ញុំពង្រីកចំនេះដឹងបន្ថែមខាងផ្នែកមុខវិជ្ជានេះ និងជាការពង្រឹងសមត្ថភាពរបស់យើងសម្រាប់ការងារផ្សេងៗទៀតទៅថ្ងៃអនាគត។

ACKNOWLEDGEMENT

ខ្ញុំសូមថ្លែងអំណរគុណនិងការដឹងគុណជូនចំពោះលោកគ្រូ ផេង ឧសភា ដែលបានផ្តល់ឱកាសដ៏មានតម្លៃក្នុងការបំពេញកិច្ចការងារនេះ ។ ក្រោយពីបានបញ្ចប់កិច្ចការងារនេះ ខ្ញុំបាទព្រមទាំងសមាជិកក្រុមទទួលបាននូវចំនេះដឹងយ៉ាងជ្រៅជ្រួតនៅក្នុងមុខវិជ្ជា ការវិភាគ និងរបរនាប្រព័ន្ធព័ត៌មាន នេះព្រមទាំងចំនេះដឹងខាងផ្នែក Programming ផងដែរ ។

លើសពីនេះទៅទៀត ខ្ញុំក៏សូមថ្លែងអំណរគុណដល់ឪពុកម្តាយ សមាជិកក្រុម និងមិត្តភក្តិរបស់ខ្ញុំទាំងអស់ដែលបានចូលរួមជាចំណែកក្នុងការជំរុញ និងផ្តល់កម្លាំងចិត្តដល់ខ្ញុំបាទរហូតដល់សម្រេចបានការងារមួយនេះ ។

ជំហានទី១៖ ការធ្វើផែនការប្រព័ន្ធ System Planning

1. មុខងារនៃប្រព័ន្ធពាណិជ្ជកម្ម Business System Function

ផ្នែកដឹកជញ្ជូនអ្នកដំណើរ Passenger Transport Department

ផ្នែកផ្ញើទំនិញ Goods Transport Department

ផ្នែកផ្តល់សេវាកម្មដល់អតិថិជន Customer Service Department

ផ្នែកបុគ្គលិក Staff Department

ផ្នែកគណនេយ្យ Accounting Department

2. ដំណើរការពាណិជ្ជកម្ម Business Operation

2.1 ផ្នែកដឹកជញ្ជូនអ្នកដំណើរ Passenger Transport Department

2.1.1 ការលក់សំបុត្រ Passenger Booking

មុខងារស្នូលនេះអនុញ្ញាតឱ្យអ្នកដំណើរកក់កៅអី ឬមកទិញសំបុត្រសម្រាប់ការធ្វើដំណើររបស់ពួកគេ ។ អតិថិជន ត្រូវផ្តល់នូវព័ត៌មានមួយចំនួនដូចជា លេខទូរស័ព្ទ គោលដៅដែលត្រូវធ្វើដំណើរទៅ និងប្រភេទរថយន្តសម្រាប់ធ្វើដំណើរ ព្រមទាំងធ្វើការបង់ប្រាក់សម្រាប់ការធ្វើដំណើរ ហើយប្រព័ន្ធនេះនឹងធ្វើការចេញវិក័យបត្រទៅវិញ។

2.1.2 ការគ្រប់គ្រងព័ត៌មានអ្នកដំណើរ Passenger Information Management

ប្រព័ន្ធនេះ កត់ត្រានិងត្រួតពិនិត្យទុកព័ត៌មានលម្អិតអ្នកដំណើរដោយសុវត្ថិភាពដែលមាន ព័ត៌មានសម្រាប់ទំនាក់ទំនង ពេលវេលាដែលបានកក់សំបុត្រនិង ចេញដំណើរ គោលដៅចេញដំណើរនិងទៅដល់ដាក់ចូលក្នុងប្រព័ន្ធ។

2.1.3 ការគ្រប់គ្រងការធ្វើដំណើរ Trip Management

មុខងារនេះតាមដានកាលវិភាគធ្វើដំណើរ កំណត់ផ្លូវក្នុងការធ្វើដំណើរជាក់លាក់និង ផ្តល់ឱ្យពួកគេនូវព័ត៌មានពាក់ព័ន្ធដូចជា ចំណុចឡើងជិះ ពេលវេលាចេញដំណើរ/ទៅដល់ ព័ត៌មានអ្នកបើកបរ និងការពន្យារពេលដែលអាចកើតមាន ។

2.1.4 ការគ្រប់គ្រងលើយានជំនិះ

គ្រប់គ្រងទៅ និងត្រួតពិនិត្យលើប្រភេទយានជំនិះដែលមានសម្រាប់អតិថិជនជ្រើសរើស និងធានាថានៅមានរថយន្តនៅទំនេរ ឬជាប់រវល់ ហើយនិងផ្តល់ព័ត៌មានកន្លែងកក់កៅអីឬទិញសំបុត្រភ្លាមៗ ដែលមិនស្មាននឹងអតិថិជនជនដទៃ។

2.2 ផ្នែកផ្ញើទំនិញ Goods Transport Department

2.2.1 ការដាក់ទំនិញផ្ញើ

ស្រដៀងទៅនឹងការកក់អ្នកដំណើរ មុខងារនេះអនុញ្ញាតឱ្យអតិថិជនដាក់បញ្ជី សម្រាប់ទំនិញរបស់ពួកគេទៅកាន់គោលដៅណាមួយដែលតម្រូវឱ្យ អតិថិជន ផ្តល់នូវព័ត៌មានមួយចំនួនដូចជា លេខទូរស័ព្ទអ្នកផ្ញើនិងទទួល គោលដៅដែលត្រូវផ្ញើទៅ និងព័ត៌មានអំពីប្រភេទទំនិញដែលត្រូវផ្ញើ ព្រមទាំងធ្វើការបង់សេវាសម្រាប់

2.2.2 ការគ្រប់គ្រងស្តុក Inventory Control

ប្រព័ន្ធនេះ ត្រួតពិនិត្យនិងរក្សាទុកត្រា ទុកព័ត៌មាននូវទំហំស្តុក ការចេញចូលទំនិញនិងត្រូវប្រាកដថាទំនិញត្រូវបានកត់ត្រានិងរក្សាទុកដោយមានរបៀបរៀបរយ។

2.2.3 ការគ្រប់គ្រងឃ្លាំង Warehouse Management

ប្រព័ន្ធនេះ កត់ត្រាទុកព័ត៌មានលម្អិតរបស់ទំនិញដោយសុវត្ថិភាពនូវ ព័ត៌មានទំនិញដែលបានដាក់ចូលឃ្លាំងដែលមានដូចជា ពេលវេលាដែលយកទំនិញចូលនិងពេលផ្ញើចេញ ប្រភេទទំនិញ បរិមាណដែលបានដាក់ចូលឃ្លាំង និងគោលដៅដែលត្រូវផ្ញើទៅ។

2.2.4 ការគ្រប់គ្រងលើការដឹកជញ្ជូន

ត្រួតពិនិត្យនិងកត់ត្រា ការវិភាគការដឹកជញ្ជូនទំនិញ និងការតាមដាននៃការដឹកជញ្ជូន និងការរៀបចំទំនិញសម្រាប់ការដឹកជញ្ជូន។ការផ្ញើនិងកត់ត្រាទុកក្នុងប្រព័ន្ធ ហើយប្រព័ន្ធនេះនិងធ្វើការចេញវិក័យបត្រទៅវិញ។

2.3 ផ្នែកផ្តល់សេវាកម្មដល់អតិថិជន Customer Service Department

2.3.1 ផ្នែកផ្តល់ព័ត៌មានលើការធ្វើដំណើរនិងការផ្ញើទំនិញ Reservation Assistance

ផ្តល់នូវការជួយបញ្ចេញមតិឱ្យអតិថិជនក្នុងការធ្វើការកក់ការធ្វើដំណើរ ឬការផ្ញើទំនិញ។

2.3.2 ការដោះស្រាយទៅលើសំនួរអតិថិជន Inquiry Handling

ផ្តល់នូវការឆ្លើយតបទៅនឹង ការសាកសួររបស់អតិថិជនតាមរយៈ លេខទូរស័ព្ទ អ៊ីមែល ឬតាមអ៊ិនធឺណែតដែលទាក់ទងទៅនឹងសេវាកម្មផ្សេងៗ តម្លៃ បទបញ្ញត្តិ របស់ក្រុមហ៊ុន។

2.3.3 ការដោះស្រាយទៅលើបញ្ហាផ្សេងៗ Problem Resolution

ជួយដោះស្រាយ ទៅលើបញ្ហាទាក់ទងនឹងសេវាកម្មដឹកជញ្ជូន ការបាត់បង់ទំនិញជាដើម រួមទាំងប្រមូលនិងវិភាគមតិកែលម្អរបស់អតិថិជនទៅលើសេវាកម្ម។

2.4 ផ្នែកបុគ្គលិក Staff Department

ផ្នែកនេះ នឹងធ្វើការកត់ត្រានូវព័ត៌មានបុគ្គលិក ដោយមានព័ត៌មានដូចជាលេខសម្គាល់ខ្លួន ឈ្មោះ លេខទូរស័ព្ទ ភេទ អាសយដ្ឋាន និងមុខដំណែងជំនាញ ត្រូវបានបញ្ចូលទៅក្នុងប្រព័ន្ធ។

2.5 ផ្នែកគណនេយ្យ Accounting Department

ផ្នែកនេះ នឹងធ្វើការកត់ត្រានិងធ្វើរបាយការណ៍នូវការចំណាយចំណូលដែលមានដូចជា ការកំណត់ តម្លៃធ្វើដំណើរនិងការផ្ញើទំនិញ ការចំណាយលើការបង់ពន្ធផ្លូវ ការជួសជុលរថយន្ត ការចំណាយរាល់ការ ដឹកជញ្ជូន ព្រមទាំងព័ត៌មាន របាយការណ៍ការបង់ប្រាក់ ។

3. បញ្ហាកើតឡើងធ្វើឱ្យមានភាពរាំងស្ទះដល់ដំណើរការពាណិជ្ជកម្ម

នៅក្នុងការដឹកជញ្ជូនអ្នកដំណើរនិងការផ្ញើទំនិញ បានរួមបញ្ចូលនូវការងារជាច្រើនផ្នែកដែលធ្វើការ ប្រទាក់ក្រឡាគ្នានិងមានភាពស្មុគស្មាញខ្លាំង វាធ្វើឱ្យមានការលំបាកក្នុងការគ្រប់គ្រងដោយមិនមាននូវ ការប្រើប្រាស់ប្រព័ន្ធកុំព្យូទ័រ។បញ្ហាទាំងនោះមាន

- ព័ត៌មានអ្នកដំណើរមិនត្រឹមត្រូវ ឬការពន្យារពេលនៃការធ្វើដំណើរ
- ភាពមិនប្រក្រតីផ្នែកបច្ចេកទេសអាចរំខានដល់ដំណើរការកក ការតាមដានពេលវេលាជាក់ស្តែង និងការចូលប្រើប្រាស់ព័ត៌មានព្រមទាំងអាច បណ្តាលឱ្យបាត់បង់ទិន្នន័យ ឬពិបាកក្នុងការស្វែងរក នូវព័ត៌មានផ្សេងៗដូចបញ្ហាណាមួយ
- ការរៀបចំនិងទុកដាក់ទំនិញមិនបានត្រឹមត្រូវក្នុងអំឡុងពេលដឹកជញ្ជូនអាចបំផ្លាញទំនិញ ឬក៏ខូច ទំនិញ
- ព្រឹត្តិការណ៍ដែលមិនបានមើលឃើញទុកជាមុន៖ ការរំខានដូចជាអាកាសធាតុអាក្រក់ គ្រោះថ្នាក់ ផ្សេងៗដែលកើតពេលធ្វើដំណើរ

4. ដំណោះស្រាយតាមបែបកុំព្យូទ័រ

ចំពោះក្រុមហ៊ុនដឹកជញ្ជូនអ្នកដំណើរនិងផ្ញើទំនិញមួយ ក្រុមហ៊ុនអាចទទួលបានការដឹកជញ្ជូនអ្នក ដំណើរនិងផ្ញើទំនិញលឿនជាងមុន។ដូច្នេះយើងត្រូវការដំណោះស្រាយតាមបែបកុំព្យូទ័រដើម្បីជួយ សម្រួលដូចជា៖

Advantages of Using Database: ចំពោះការប្រើប្រាស់ប្រព័ន្ធព័ត៌មាន យើងអាចកាត់បន្ថយរយៈពេល នៃ ការស្វែងរកកន្លែងផ្ទុកទិន្នន័យ និងភាពភ័យច្រឡំនៃទិន្នន័យបានស្មើតែទាំងស្រុង ពីព្រោះទិន្នន័យ ទាំងអស់ ត្រូវបានផ្ទុកនៅក្នុង Database ដែលផ្តល់ឱកាសងាយស្រួលដល់អ្នកគ្រប់គ្រង។

Integration of Data: ការងារទាំងអស់ប្រទាក់ក្រឡាគ្នានិងពឹងផ្អែកលើផ្នែកនីមួយៗទៅវិញទៅមក។

Expandable: ងាយស្រួលក្នុងការពង្រីកទំហំនៃការងារនៅថ្ងៃអនាគត។

Less Paper-based Documents: កាត់បន្ថយការប្រើប្រាស់ឯកសារជាទំរង់ក្រដាស។

5. ស្ថាបត្យកម្មនៃប្រព័ន្ធព័ត៌មាន

ជាសេចក្តីផ្តើមនៃការស្ថាបនាប្រព័ន្ធព័ត៌មានយើងត្រូវយល់ដឹងពីរចនាសម្ព័ន្ធនៃការប្រើប្រាស់ប្រព័ន្ធព័ត៌មាននេះជាមុនសិន។ នៅក្នុងប្រព័ន្ធព័ត៌មាន SQL Server នឹងត្រូវបានប្រើដើម្បីផ្ទុកទិន្នន័យហើយ C# Programming នឹងត្រូវបានប្រើដើម្បីបង្កើត Interfaces សំរាប់បញ្ចូលទិន្នន័យក្នុងផ្នែកនីមួយៗ។

1. តម្រូវការផ្នែករឹង (Hardware Requirements)

ចំពោះ Server Computer

- AMD or Intel-based x64 Processors
- 8 GB system memory
- CPU Clock Speed: x64 2.0 GHz or more
- Data Quality Services requires an additional 2 GB of system memory to operate optimally.
- Hard Disk: 3TB

ចំពោះ Client Computer

- 64-bit Intel® Pentium® 4 or compatible processor running at 2 GHz or greater
- 8 GB RAM
- Graphics card: NVIDIA
- Disk space: 256 GB for client components

2. តម្រូវការផ្នែកទន់ចំពោះ Server (Server Software Requirements)

- .NET edition 6.0 is required for running SQL Server 2019
- 6 GB of hard disk space is needed for a full installation.
- Windows Server 2016 or 2019 Editions only.

6. ការកំណត់ពេលវេលាដែលត្រូវចំណាយលើព្រំដែនការងារ

ជំហាន	ដំណាក់កាលនៃការងារ	ថ្ងៃចាប់ផ្តើម	ថ្ងៃបញ្ចប់
ដំណាក់កាលទី១	ការធ្វើផែនការប្រព័ន្ធ (System Planning)	១១ ខែមីនា ឆ្នាំ២០២៤	២៥ មីនា ឆ្នាំ២០២៤
ដំណាក់កាលទី២	ការវិភាគប្រព័ន្ធ (System Analysis)	២៥ ខែមីនា ឆ្នាំ២០២៤	២៤ ខែមេសា ឆ្នាំ២០២៤
ដំណាក់កាលទី៣	ការរចនាប្រព័ន្ធ (System Design)	២៤ ខែមេសា ឆ្នាំ២០២៤	១៣ ខែឧសភា ឆ្នាំ២០២៤
ដំណាក់កាលទី៤	ការស្ថាបនាប្រព័ន្ធ (System Implementation)	១៣ ខែឧសភា ឆ្នាំ២០២៤	១៧ ខែមិថុនា ឆ្នាំ២០២៤

7. ការកំណត់ថវិកាដែលត្រូវចំណាយលើព្រំដែនការងារ

→ចំណាយលើផ្នែករឹង

Hardware			
	Price Per Unit	Quantity	Sub Total
Dell PowerEdge R730 Server (Xeon E5-2620 / 16GB / 3T)	\$3,189.00	1	\$3,189.00
Dell OptiPlex , Intel Core i5 3th 3.2 GHz, 16GB RAM, HDD, New 22 Inch ,Windows 10	\$217	6	\$1,302.00
POWER T UPS 1KVA	\$185.00	5	\$925.00
Computer Networking (Accessories Included)	\$360.00	1	\$360.00
Epson EcoTank ET-M3170 Wireless Monochrome All-in-One Supertank Printer, Plus ADF, Fax and Ethernet	\$350.00	2	\$700.00
Total			\$6,476.00

→ចំណាយលើផ្នែកទន់

Software			
SQL Server Standard Edition	\$3,717.00	1	\$3,717.00
Windows 10 Pro	\$200.00	6	\$1,200.00
SAP Crystal Reports	\$495.00	1	\$495.00
Windows Server 2019 Essentials Edition	\$501.00	1	\$501.00
Total			\$ 5,913.00

ដូចនេះការចំណាយលើ Hardware និង Software សរុបអស់ 12,389 សម្រាប់ប្រព័ន្ធការដឹកជញ្ជូន និងផ្ញើទំនិញ។

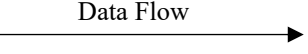
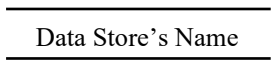
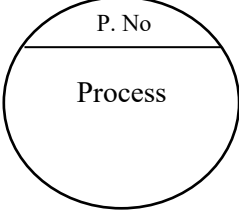
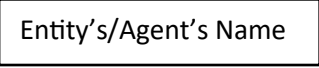
ជំហានទី២ ៖ ការធ្វើការវិភាគប្រព័ន្ធ System Analysts

I. ការធ្វើ Process Modeling

Process Modeling គឺជាបច្ចេកទេសមួយដែលត្រូវបានប្រើសំរាប់រៀបរាប់ពីដំណើរការរបស់ប្រព័ន្ធព័ត៌មានដូចជាការបញ្ចូលព័ត៌មាន ការប្រតិបត្តិទិន្នន័យ និងការបញ្ចេញលទ្ធផលព័ត៌មានតាមរយៈការគូស Data Flow Diagram ការសរសេរ Structured English និងការគូស Decision Tree និង Decision Table ។ Process Modeling ត្រូវបានបង្កើតឡើងសំរាប់ធ្វើជាឯកសារទុកឲ្យ Programmer ដើម្បីយកទៅប្រើប្រាស់ក្នុងការសរសេរប្រកាម ។

1.Data Flow Diagram

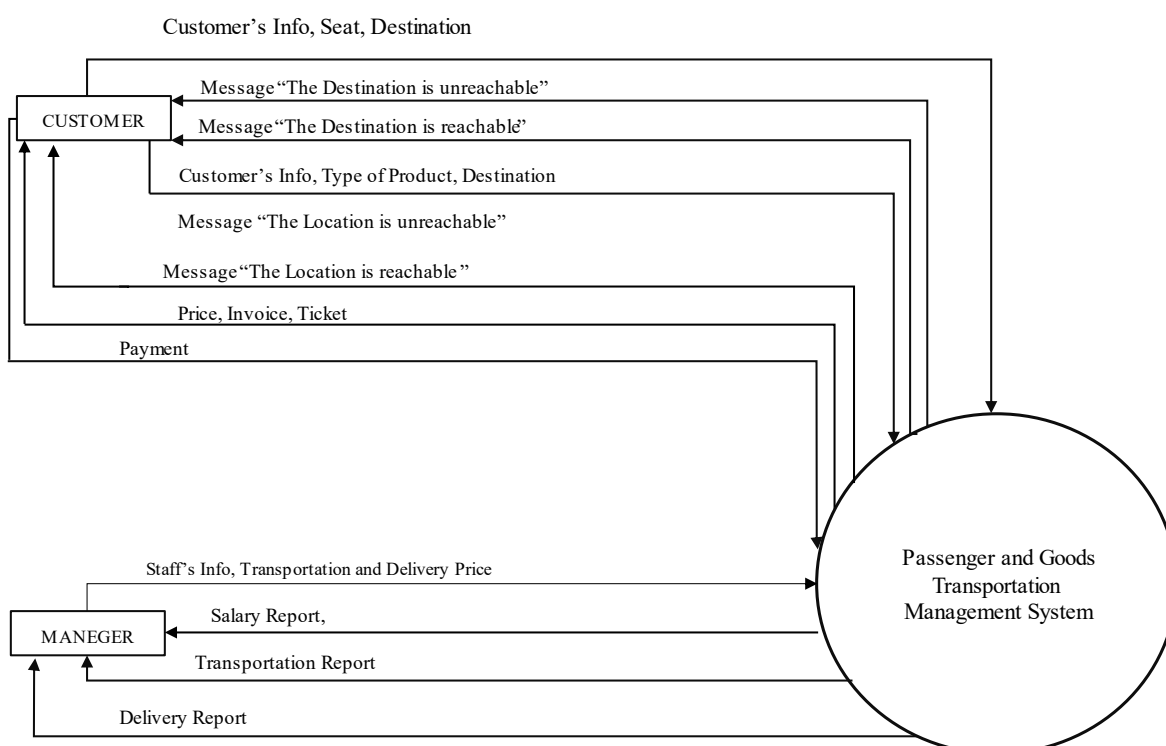
ដ្យាក្រាមលំហូរទិន្នន័យ (Data Flow Diagram ឬ DFD) គឺជាឧបករណ៍ដែលត្រូវបានប្រើក្នុងការកំណត់ទ្រង់ទ្រាយនៃដំណើរប្រតិបត្តិ ដោយពិពណ៌នាលំហូរទិន្នន័យនិងដំណើរការប្រតិបត្តិទាំងឡាយដែលស្ថិតនៅខាងក្រៅនិងខាងក្នុងប្រព័ន្ធព័ត៌មាន។ នៅក្នុងការដំណាក់កាលវិភាគប្រព័ន្ធនេះ យើងបានជ្រើសរើសយកនិមិត្តសញ្ញារបស់ DeMarco-Yourdon ដើម្បីគូស Data Flow Diagram ។

DeMarco-Yourdon	សមាសធាតុ	អត្ថន័យរបស់សមាសធាតុ
	Data Flow	ខ្សែប្រព្រឹត្តិសំរាប់បញ្ជូនទិន្នន័យពីសមាសធាតុមួយទៅកាន់សមាសធាតុមួយទៀត។
	Data Store	កន្លែងផ្ទុកទិន្នន័យជានិរន្តរបស់ប្រព័ន្ធព័ត៌មានដោយវានឹងក្លាយជា File ឬ Table ។
	Process	គឺជាសកម្មភាពឬការប្រតិបត្តិក្នុងការបំពេញ លែងទិន្នន័យពីការបញ្ចូលទៅការបញ្ចេញ។វានឹងក្លាយជា Sub Procedure ឬ Function ។
	Entity/Agent	ជាមនុស្សម្នាក់ឬកន្លែងណាមួយដែលមានទំនាក់ទំនងជាមួយប្រព័ន្ធព័ត៌មានដូចជាផ្តល់ព័ត៌មានទៅឲ្យប្រព័ន្ធព័ត៌មាន ឬទទួលលទ្ធផលជាព័ត៌មានចេញពីប្រព័ន្ធព័ត៌មាន វិញ។

2. ប្រភេទនៃដ្យាក្រាមលំហូរទិន្នន័យ

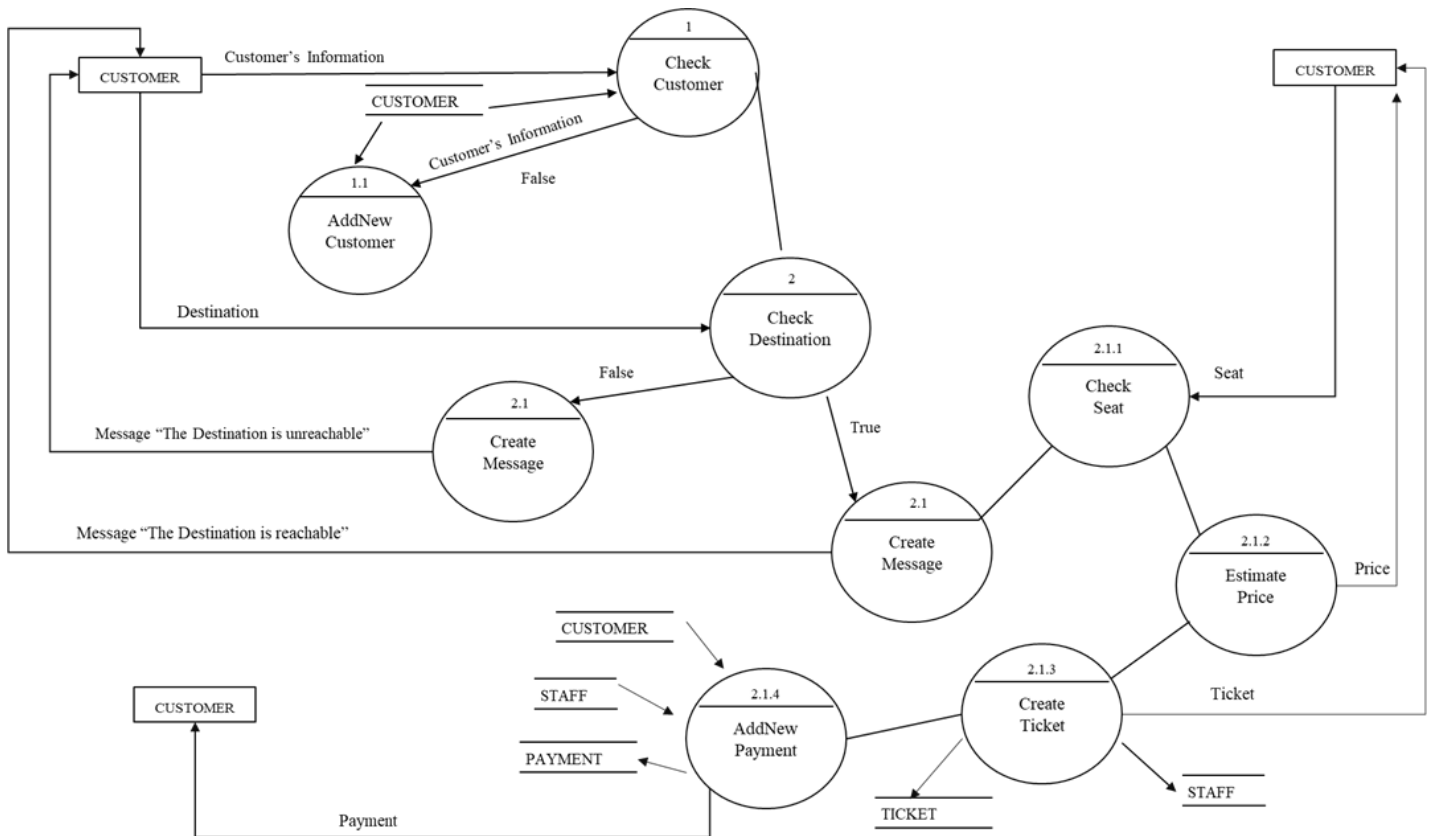
2.1.Context Diagram

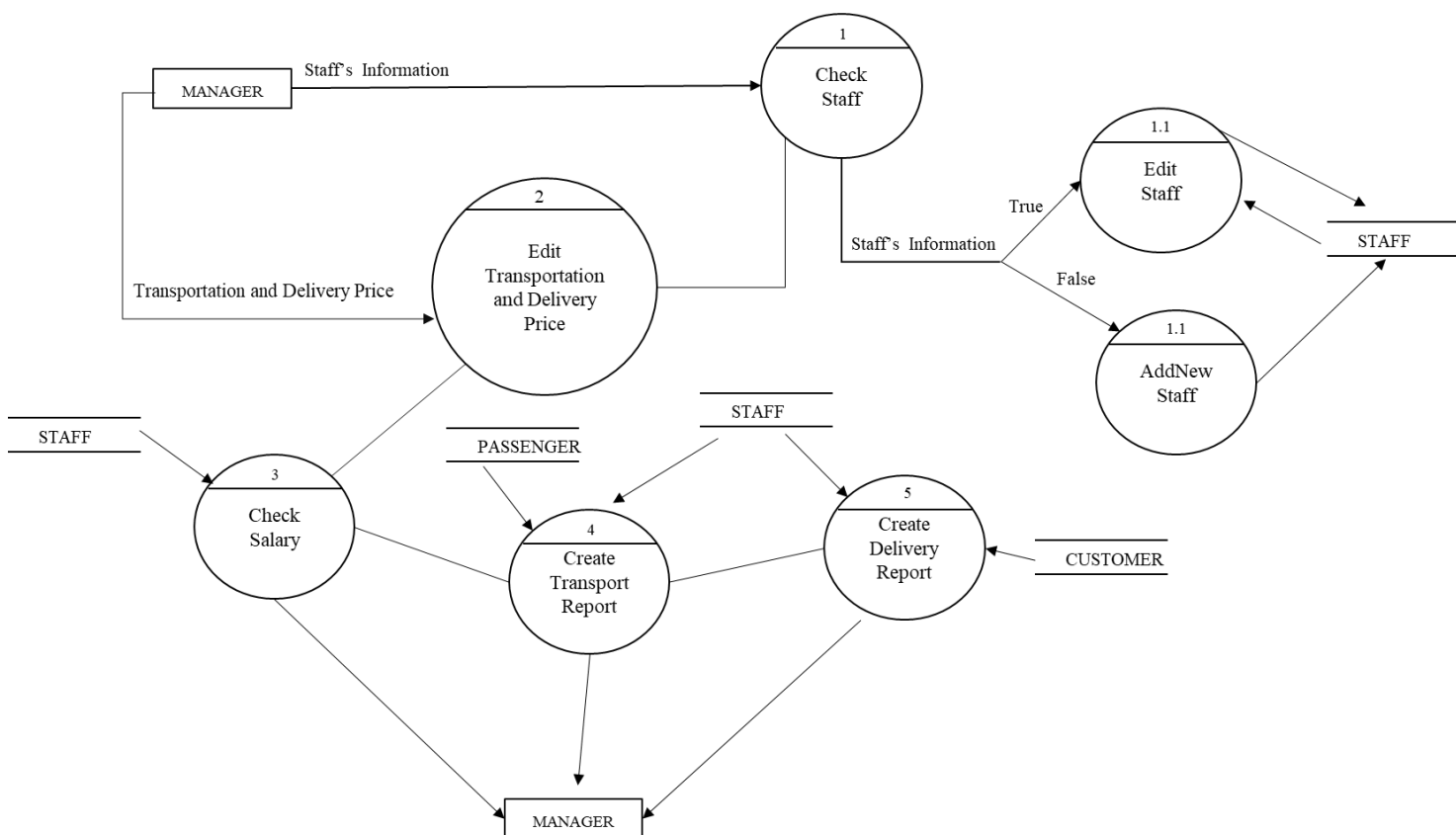
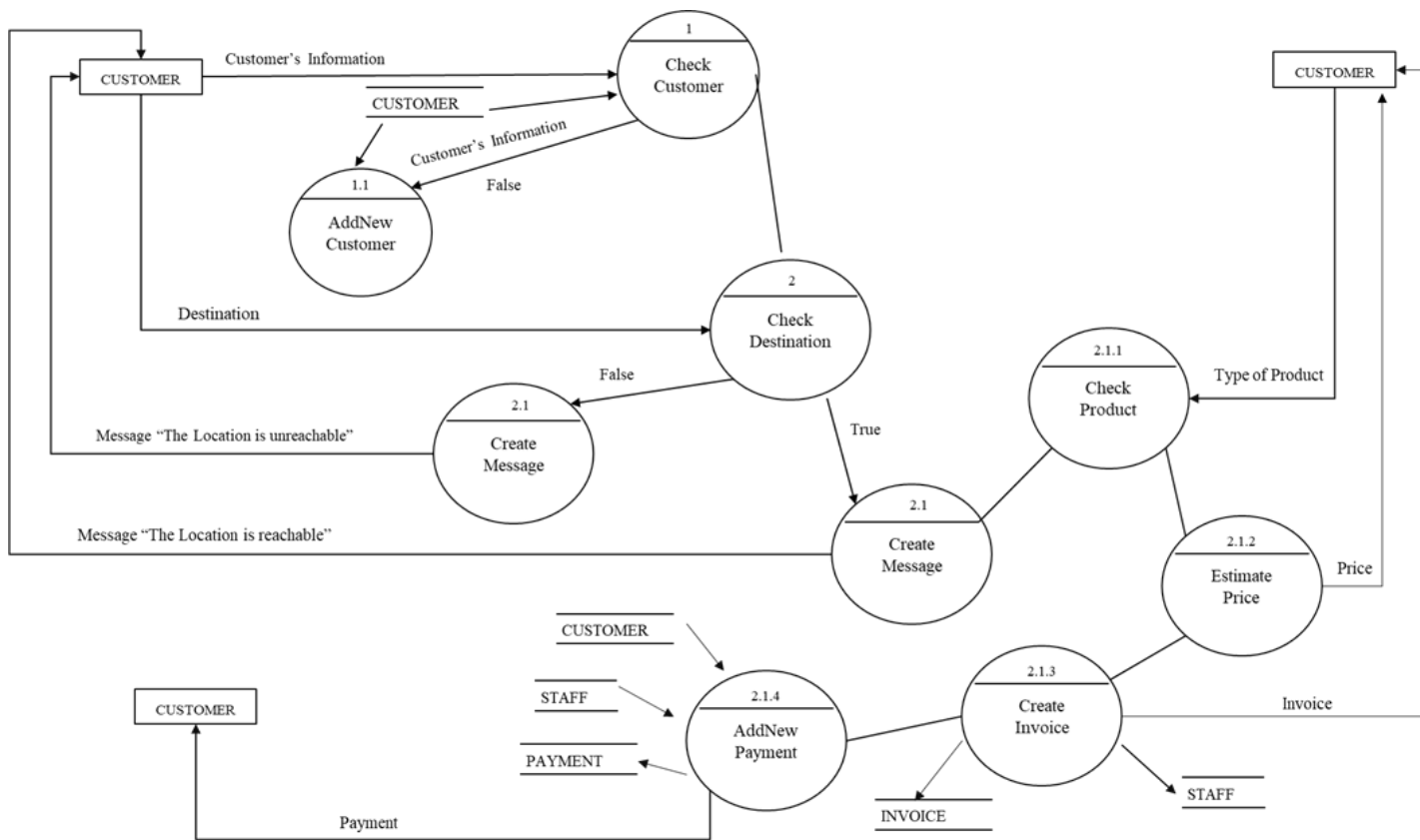
Context Diagram គឺជាដ្យាក្រាមដែលបង្ហាញនូវដំណើរការរបស់ប្រព័ន្ធព័ត៌មានតាមរយៈការសង្កេតមើលពីខាងក្រៅ។ លក្ខណៈរបស់វាគឺវាមាន Process តែមួយគត់ដែលតំណាងឲ្យប្រព័ន្ធព័ត៌មានទាំងមូល។ វាមាន Agents ទាំងឡាយដែលមានទំនាក់ទំនងជាមួយប្រព័ន្ធព័ត៌មានក្នុងការផ្តល់ព័ត៌មានទៅឲ្យប្រព័ន្ធព័ត៌មានឬទទួលបានលទ្ធផលប្រព័ន្ធព័ត៌មានវិញ។ Context Diagram របស់ប្រព័ន្ធគ្រប់គ្រងការដឹកជញ្ជូនអ្នកដំណើរនិងការផ្ញើឥវ៉ាន់៖



2.2. Physical Diagram

Physical Level Diagram គឺជាដ្យាក្រាមដែលបង្ហាញសកម្មភាពទាំងអស់របស់ប្រព័ន្ធព័ត៌មាន។ លក្ខណៈរបស់វាគឺវាមាន Processes ទាំងឡាយណាដែលតំណាងអោយសកម្មភាពនៃដំណើរប្រតិបត្តិក្នុងប្រព័ន្ធព័ត៌មានព្រមទាំង Data Stores និង Agents ទាំងឡាយដែលមានទំនាក់ទំនងជាមួយនឹង Processes ទាំងនោះតាមរយៈ Data Flow។





I. ការធ្វើ Data Modeling

បន្ទាប់ពីការសិក្សាលើ Process Modeling បានបញ្ចប់យើងចាប់ផ្តើមសិក្សាលើ Data Modeling វិញ ម្តង ។ Data Modeling គឺជាបច្ចេកទេសតម្រង់ទិសទិន្នន័យ (Data-oriented technique) ដែលត្រូវប្រើសម្រាប់ ប្រមូលយកព័ត៌មានទាំងឡាយទៅនិងមុខងារនៃប្រព័ន្ធពាណិជ្ជកម្ម ។ ព័ត៌មានទាំងអស់នោះមានដូចជា៖ ព័ត៌មានមនុស្ស វត្ថុ ទីកន្លែង និងព្រឹត្តិការណ៍ដែលត្រូវការកត់ត្រាទុក ។ ជំហានសំខាន់របស់ Process Modeling មានដូចជា

1. កំណត់ Entity Set

Entity Set គឺជាសំណុំនៃ Entities ទាំងឡាយណាដែលមានប្រភេទដូចគ្នា (Entity គឺជាព័ត៌មាន មនុស្ស វត្ថុ ទីកន្លែងនិង ព្រឹត្តិការណ៍) ។ Entity Set ចែកចេញជាពីរគឺ៖ Strong Entity Set និង Weak Entity Set ។ Strong Entity Set គឺជា Entity Set ដែលមិនអាស្រ័យទៅនឹង Entity Set ផ្សេងទៀត ។ Weak Entity Set គឺជា Entity Set ដែលអាស្រ័យនឹង Entity Set ដទៃទៀត ។

យោងតាមគោលការណ៍នៃការគ្រប់គ្រង យើងកំណត់ Entity Set ចំនួន ១១ ដូចជា

PASSENGER, CUSTOMER, STAFF, ITEMS, ROUTE, VEHICLE, BOOKING, PAYMENT, TRANSPORT, INVOICE, USER, TICKET

2. កំណត់ Strong Entity Set and Weak Entity Set

-Strong Entity Set

CUSTOMER

STAFF

ROUTE

VEHICLE

-Weak Entity Set

ITEMS

BOOKING

PAYMENT

TRANSPORT

INVOICE

TICKET

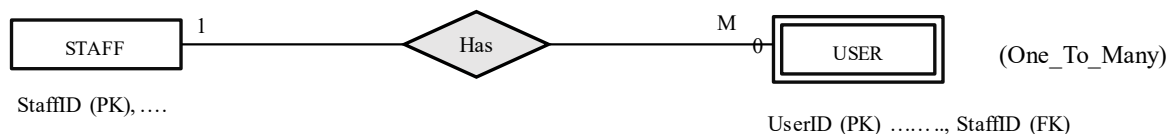
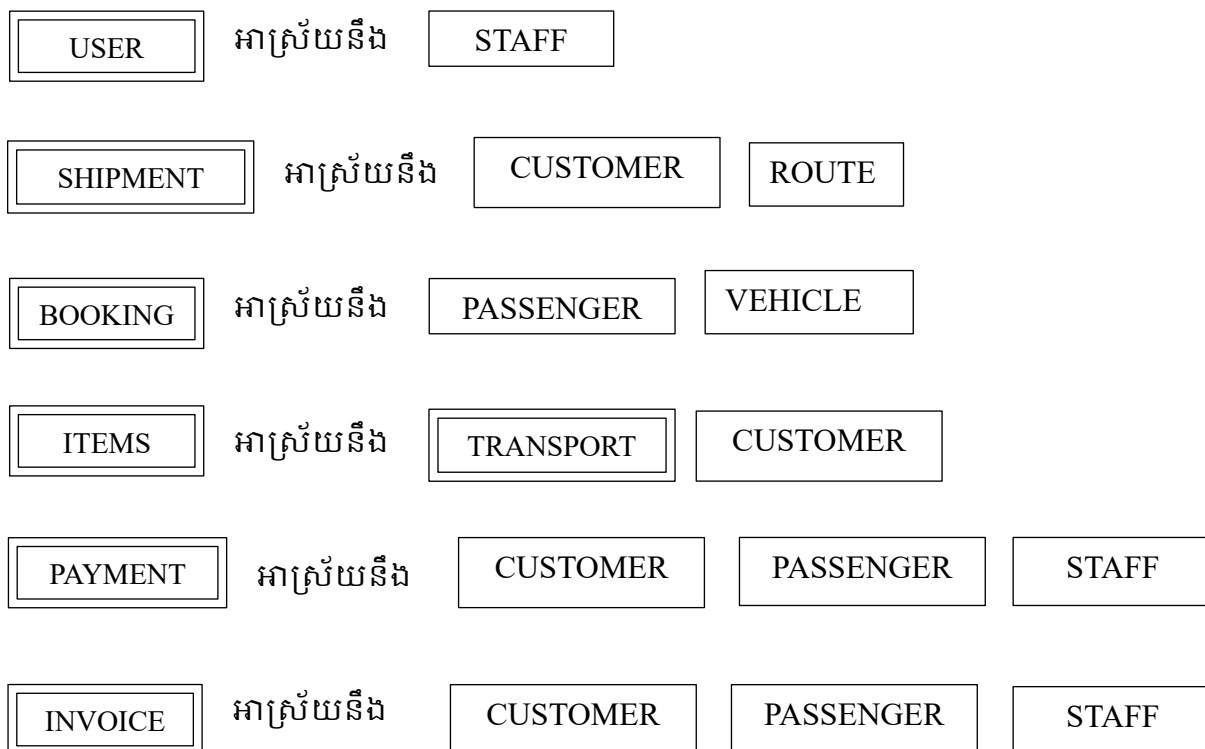
USER

3.កំណត់Attribute and Primary Key

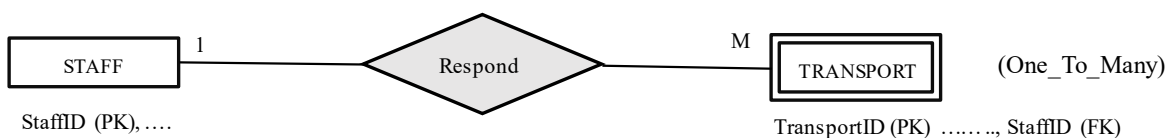
Entity Set	Attribute	បំណកស្រាយ
(S) CUSTOMER	CustomerID (PK)	ជាលេខកូដសម្គាល់អតិថិជន
	Name	ជាឈ្មោះអតិថិជន
	Age	ជាកេរអតិថិជន
	Gender	ជាលេខទូរស័ព្ទអតិថិជន
	ContactInfo	ជាលេខកូដសម្គាល់របស់បុគ្គលិក។
(S) STAFF	StaffID (PK)	ជានាមត្រកូលរបស់បុគ្គលិក។
	FirstName	ជានាមឈ្មោះរបស់បុគ្គលិក។
	LastName	ជាកេររបស់បុគ្គលិក។
	Gender	ជាថ្ងៃខែឆ្នាំកំណើតរបស់បុគ្គលិក។
	BirthDate	ជាតួនាទីរបស់បុគ្គលិក។
	Position	ជាលេខផ្ទះរបស់បុគ្គលិក។
	HouseNo	ជាលេខផ្លូវនៃអាសយដ្ឋានរបស់បុគ្គលិក។
	StreetNo	ជាសង្កាត់ដែលបុគ្គលិករស់នៅ។
	Sangkat	ជាខណ្ឌដែលបុគ្គលិករស់នៅ។
	Khann	ជាខេត្តឬក្រុងដែលបុគ្គលិករស់នៅ។
	Provice/City	ជាលេខទំនាក់ទំនងក្នុងក្រុមហ៊ុនរបស់បុគ្គលិក។
	ContacNumber	ជាលេខទំនាក់ទំនងផ្ទាល់ខ្លួនរបស់បុគ្គលិក។
	PersonalNumber	ជាប្រាក់ខែរបស់បុគ្គលិក។
	Salary	ជាថ្ងៃចូលធ្វើការរបស់បុគ្គលិក។
	HireDate	ជាលេខកូដសម្គាល់អតិថិជន
	Photo	ជារូបថតរបស់បុគ្គលិក។
	StopWork	សម្គាល់ថាបុគ្គលិកឈប់ធ្វើការឬនៅ។
(S) ROUTE	RouteID (PK)	ជាលេខសម្គាល់គោលដៅ
	Origin	ជាកន្លែងចាប់ផ្តើម
	Destination	ជាគោលដៅត្រូវទៅ
	Distance	ជាចម្ងាយរបស់ផ្លូវ
(S) VEHICLE	VehicleID (PK)	ជាលេខសម្គាល់រថយន្ត
	Type	ជាប្រភេទរថយន្ត
	SeatNum	ជាលេខរបស់កៅអី
	Capacity	ជាទំហំផ្ទុករបស់រថយន្ត

	Lincense_Plate	ជាស្លាកលេខរបស់រថយន្ត
	Status	ជាស្ថានភាពរថយន្ត
(W) ITEM	ItemID (PK)	ជាលេខសម្គាល់ទំនិញ
	Category	ជាប្រភេទទំនិញ
	Description	ជាការពណ៌នាអំពីទំនិញ
	Weight	ជាទម្ងន់របស់ទំនិញ
(W) BOOKING	BookingID (PK)	ជាលេខសម្គាល់ការកក់
	Date	ជាកាលបរិច្ឆេទការកក់
	TotalPaid	ជាចំនួនទឹកប្រាក់ការកក់
(W) PAYMENT	PaymentID	ជាលេខសម្គាល់ការបង់ប្រាក់
	PayDate	ជាកាលបរិច្ឆេទការបង់ប្រាក់
	PaidAmout	ជាចំនួនទឹកប្រាក់ដែលបានបង់
(W) TRANSPORT	TransportID (PK)	លេខសម្គាល់ការដឹកជញ្ជូន
	TransportDate	ជាកាលបរិច្ឆេទការដឹកជញ្ជូន
	Status	ជាស្ថានភាពនៃការដឹកជញ្ជូន
(W) INVOICE	InvoiceID (PK)	ជាលេខសម្គាល់វិក័យប័ត្រ
	InvoiceDate	ជាកាលបរិច្ឆេទការចេញវិក័យប័ត្រ
	InvoiceAmount	ជាចំនួនទឹកប្រាក់សរុបដែលបានបង់
	PaidAmout	ជាចំនួនទឹកប្រាក់ដែលបានបង់
(W) USER	UserID	ជាលេខសម្គាល់របស់អ្នកប្រើប្រាស់
	UserName	ជាឈ្មោះរបស់អ្នកប្រើប្រាស់
	Password	ជាលេខសម្ងាត់របស់អ្នកប្រើប្រាស់
(W) Ticket	TicketID	ជាលេខសម្គាល់របស់សំបុត្រ
	Price	ជាតម្លៃសំបុត្រ
	TicketDate	ជាកាលបរិច្ឆេទរបស់សំបុត្រ

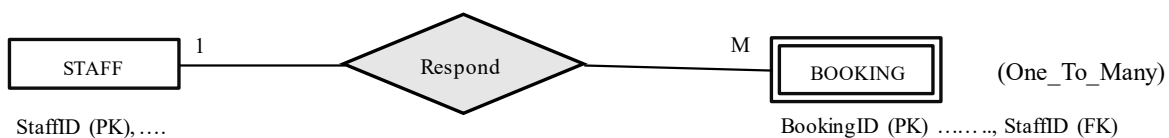
4.កំណត់ Relationship Entity Set



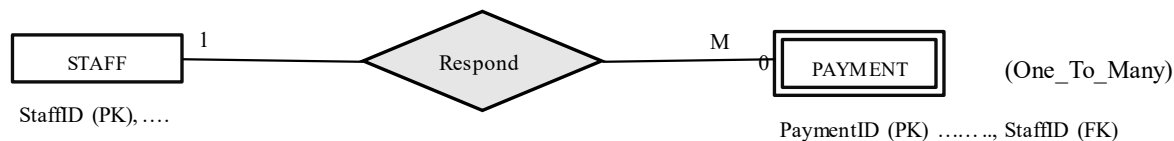
- (a) USER នីមួយៗគឺជាអតិថិជនរបស់ STAFF តែម្នាក់គត់។
(b) STAFF ម្នាក់ៗអាចអត់ឬមាន USER មួយឬច្រើន។



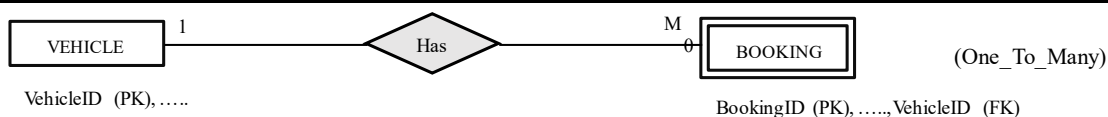
- (a) TRANSPORT ម្តងៗត្រូវបានទទួលខុសត្រូវដោយ STAFF តែម្នាក់គត់។
(b) STAFF ម្នាក់ៗទទួលខុសត្រូវលើការTRANSPORT មួយឬច្រើនដង។



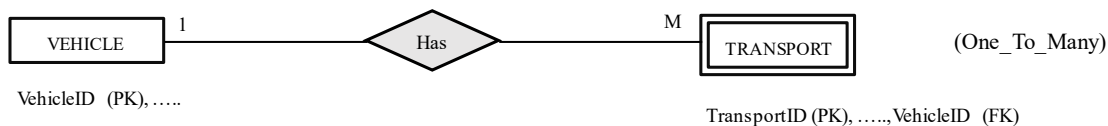
- (a) BOOKING ម្តងៗត្រូវបានទទួលខុសត្រូវដោយ STAFF តែម្នាក់គត់។
(b) STAFF ម្នាក់ៗទទួលខុសត្រូវលើការBOOKING មួយឬច្រើនដង។



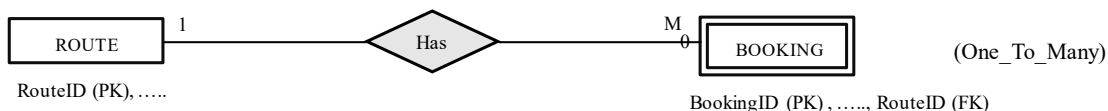
- (a) PAYMENT ម្តងៗត្រូវបានទទួលខុសត្រូវដោយ STAFF តែម្នាក់គត់។
(b) STAFF ម្នាក់ៗអាចអត់ឬទទួលខុសត្រូវលើការPAYMENT មួយឬច្រើនដង។



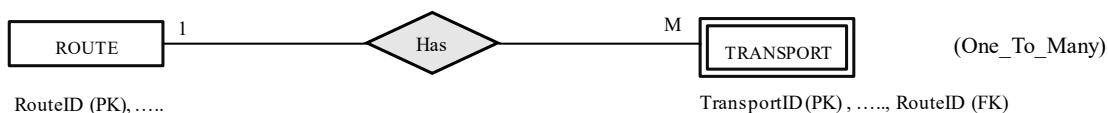
- (a) BOOKING ម្តងៗមាន VEHICLE តែមួយគត់។
 (b) VEHICLE មួយអាចអត់ឬមាន BOOKING មួយឬច្រើនដង។



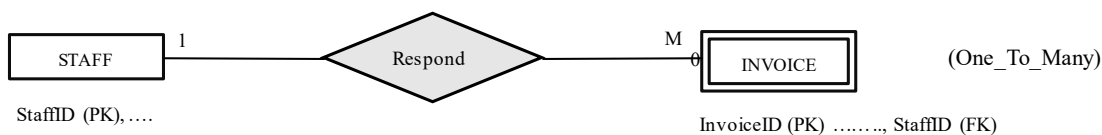
- (a) TRANSPORT ម្តងៗមាន VEHICLE តែមួយគត់។
 (b) VEHICLE មួយមាន TRANSPORT មួយឬច្រើនដង។



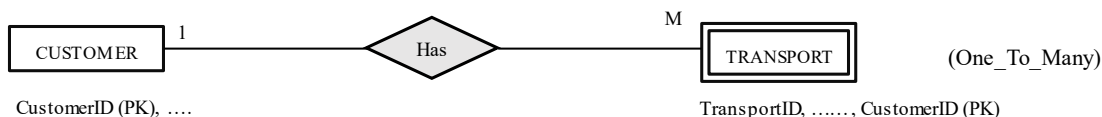
- (a) BOOKING ម្តងៗមាន ROUTE តែមួយគត់។
 (b) ROUTE មួយអាចអត់ឬមាន BOOKING មួយឬច្រើនដង។



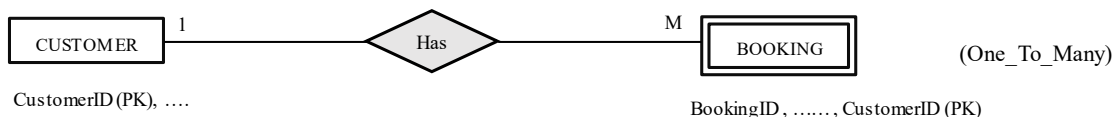
- (a) TRANSPORT ម្តងៗមាន ROUTE តែមួយគត់។
 (b) ROUTE មួយមាន TRANSPORT មួយឬច្រើនដង។



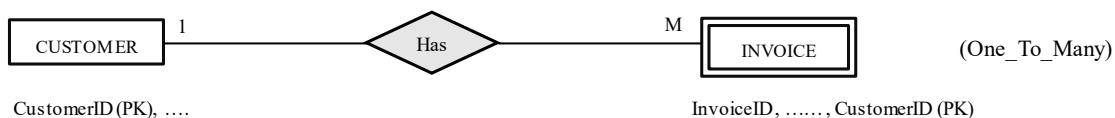
- (a) INVOICE នីមួយៗត្រូវបានទទួលខុសត្រូវដោយ STAFF តែម្នាក់គត់។
 (b) STAFF ម្នាក់ៗអាចអត់ឬទទួលខុសត្រូវលើ INVOICE មួយឬច្រើន។



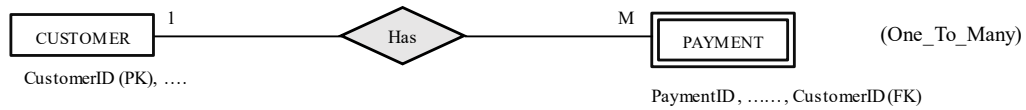
- (a) TRANSPORT ម្តងៗជាប់សំ CUSTOMER តែម្នាក់គត់។
 (b) CUSTOMER ម្នាក់ៗអាចមាន TRANSPORT មួយឬច្រើនដង។



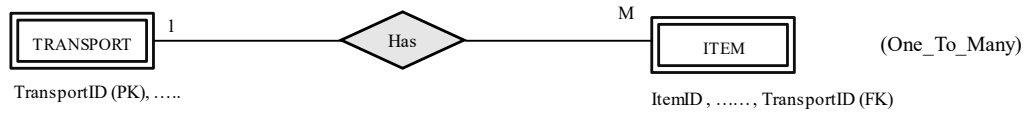
- (a) BOOKING ម្តងៗជាប់សំ CUSTOMER តែម្នាក់គត់។
 (b) CUSTOMER ម្នាក់ៗអាចមាន BOOKING មួយឬច្រើនដង។



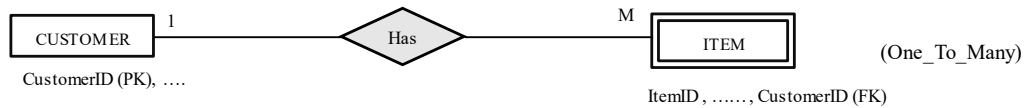
- (a) INVOICE នីមួយៗជាប់សំ CUSTOMER តែម្នាក់គត់។
 (b) CUSTOMER ម្នាក់ៗអាចមាន INVOICE មួយឬច្រើន។



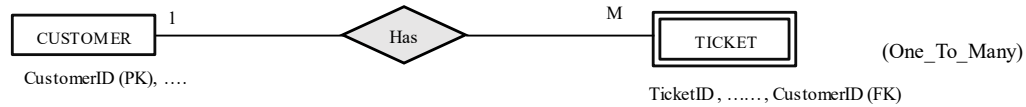
- (a) PAYMENT នីមួយៗជាប់សំ CUSTOMER តែម្នាក់គត់។
 (b) CUSTOMER ម្នាក់ៗអាចមាន PAYMENT មួយឬច្រើន។



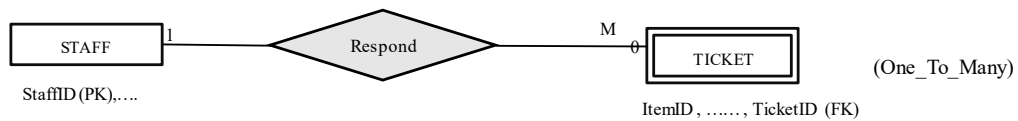
- (a) ITEM នីមួយៗមាន TRANSPORT ម្តង។
 (b) TRANSPORT ម្តងៗអាចមាន ITEM មួយឬច្រើន។



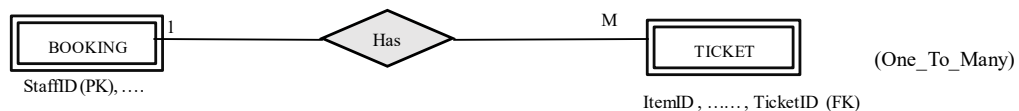
- (a) ITEM នីមួយៗជាប់សំ CUSTOMER តែម្នាក់។
 (b) CUSTOMER ម្នាក់ៗអាចមាន ITEM មួយឬច្រើន។



- (a) TICKET នីមួយៗជាប់សំ CUSTOMER តែម្នាក់។
 (b) CUSTOMER ម្នាក់ៗអាចមាន TICKET មួយឬច្រើន។



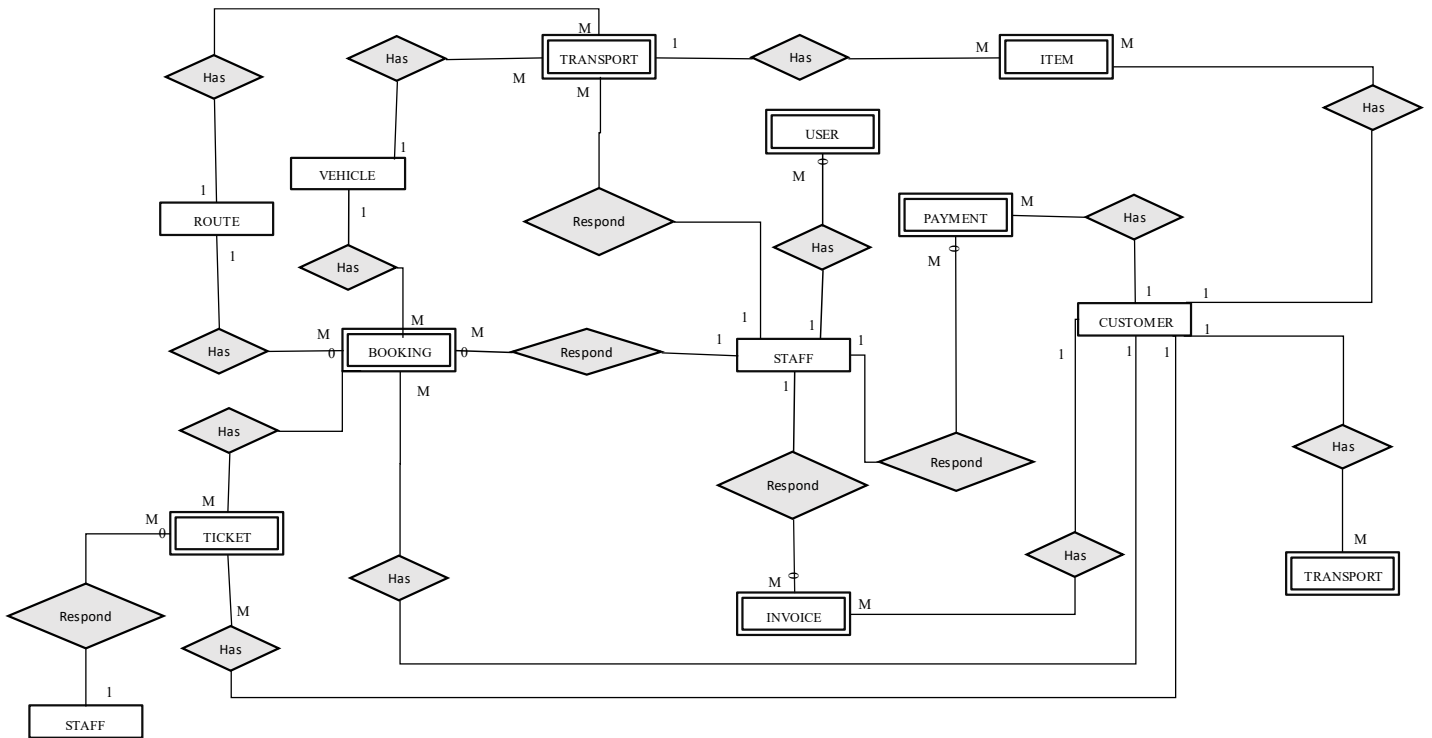
- (a) TICKET នីមួយៗ ត្រូវបានទទួលខុសត្រូវដោយ STAFF តែម្នាក់។
 (b) STAFF ម្នាក់ៗ អាចអត់ឬទទួលខុសត្រូវលើ TICKET មួយឬច្រើន។



- (a) TICKET នីមួយៗត្រូវបាន BOOKING ម្តង។
 (b) BOOKING ម្តងៗមាន TICKET មួយឬច្រើន។

5. Entity Relationship Diagram ERD

ក្រោយពីយើងធ្វើទំនាក់ទំនងរវាង Entity Sets និងបំប្លែងប្រភេទ Many-To-Many ទៅ One-To-Many រួច យើងអាចគូស ERD (Entity-Relationship-Diagram) ។ តាមរយៈ ERD នោះ យើងអាចកំណត់ Tables ទាំងអស់របស់ Database Structures ក្នុងទំរង់ Relational Schema



6. កំណត់ Table

ក្រោយពីបានគូស Relationship រវាង Entity Sets ព្រមទាំង Entity Relationship Diagram រួចមក យើងអាចកំណត់ table មួយចំនួនដូចជា៖

tbCustomer (**CustomerID (PK)**, Name, Age, Gender, ContactInfo)

tblStaff (**StaffID (PK)**, FirstName, LastName, Gender, BirthDate, Position, HouseNo, StreetNo, Sangkat, Khann, ProvinceCity, ContactNumber, PersonalNumber, Salary, HiredDate, Photo, StoppedWork)

tbRoute (**RouteID (PK)**, Origin, Destination, Distance)

tbVehicle (**VehicleID (PK)**, Type, Capacity, LincsesPlate)

tblItem(**ItemID (PK)**, Category, Description, Weight, **CustomerID (FK)**, **TransportID (FK)**)

tbBooking (**BookingID (PK)**, Date, Status, **CustomerID (FK)**, **VehicleID (FK)**)

tbTransport (**TransportID (PK)**, DeliveryDate, Status, **CustomerID (FK)**, **RouteID (FK)**)

tbPayment (**PaymentID (PK)**, PayDate, PaidAmount, **CustomerID (FK)**, **StaffID (FK)**)

tbInvoice (**InvoiceID (PK)**, InvoiceDate, InvoiceAmount, PaidAmount, **CustomerID (FK)**, **StaffID (FK)**)

tbUser (**UserID (PK)**, UserName, Password, **StaffID (FK)**)

tbTicket(**TicketID (PK)**, SeatNum ,Date, Price,**CustomerID (FK)**, **StaffID (FK)**)

7. De-normalization

តាមរយៈ Table ដែលបានកំណត់ខាងលើ យើងត្រូវធ្វើ De-normalization ទៅលើ Tables មួយចំនួន ដោយទទួលបាន៖

tblUser (**UserID(PK)**, Username, Password, **StaffID (FK)**, StaffName, StaffPosition)

tblTransport (**Transport ID(PK)**, TrasportDate, Status, **CostomerID(FK)**, CosName, CosContactinfo, **TargetID (FK)**, Origin, Destination)

tblBooking (**BookingID (PK)**, Date, Status, **CostomerID(FK)** CosName, CosContactInfo, **VehicleID(FK)**, Type, **TargetID**, Destination)

tblItem (**ItemID (PK)**, Category, Description, weight, **TransportID(FK)**, TransportDate, Status, **CostomerID(FK)** CosName, CosContactinfo)

tblPayment (**Payment ID (PK)**, PayDate, PaidAmout, **CostomerID (FK)**,CosName, CosContactInfo, **StaffID(FK)**, StaffName, StaffPosition)

tblInvoice (**InvoiceID (PK)**, InvoiceDate, InvoiceAmount, PaidAsout, **CostomerID (FK)** CosName, CosContactInfo, **StaffID (FK)**, StaffName, StaffPosition)

tblTicket (**TicketID (PK)**, Date, Price, **CostomerID (FK)** CosName, CosContactInfo, **StaffID (FK)**, StaffName, StaffPosition,**BookingID**,BookingDate)

ជំហានទី៣ ៖ ការធ្វើការចងប្រព័ន្ធ System Design

ការចងប្រព័ន្ធ (System Design) ស្ថិតក្នុងដំណាក់កាលទី ៣ នៃវដ្តជីវិតនៃការអភិវឌ្ឍន៍ប្រព័ន្ធដែលមានអ្នកចូលរួមដូចជាអ្នកវិភាគ និងអ្នករចនាប្រព័ន្ធដោយមានសកម្មភាពដូចជា៖ ការចង Data Dictionary និង ការរចនា User Interface Layouts ។

I. ការរចនា Data Dictionary

Data Dictionary ត្រូវបានបង្កើតឡើងសំរាប់ឲ្យ Programmer យកទៅប្រើប្រាស់ក្នុងការបង្កើត Tables ទាំងនោះនៅក្នុង Database នៃ RDBMS ដែលពួកគេបានប្រើ ។

1. tbCustomer

Key	Column Name	ប្រភេទទិន្នន័យ	Allow Null
PK	CustomerID	int	
	Name	varchar(50)	
	Age	tinyint	
	Gender	char(1)	
	ContactInfo	varchar(20)	

2. tbStaff

Key	Column Name	ប្រភេទទិន្នន័យ	Allow Null
PK	StaffID	int	
	FirstName	nvarchar(50)	
	LastName	nvarchar(50)	
	Gender	char(1)	
	BirthDate	date	
	Position	varchar(100)	
	HouseNo	nvarchar(15)	
	StreetNo	nvarchar(25)	
	Sangkat	nvarchar(50)	
	Khann	nvarchar(50)	
	Province/City	nvarchar(50)	
	ContactNumber	varchar(20)	
	PersonalNumber	varchar(20)	
	Salary	Money	
	HireDate	date	
	Photo	varbinary(max)	
	StopWork	bit	

3.tbRoute

Key	Column Name	ប្រភេទទិន្នន័យ	Allow Null
PK	RouteID	int	
	Origin	varchar(50)	
	Destination	varchar(50)	
	Distance	smallint	

4.tbVehicle

Key	Column Name	ប្រភេទទិន្នន័យ	Allow Null
PK	VehicleID	int	
	Type	varchar(50)	
	SeatNum	smallint	
	Capacity	varchar(50)	
	Lincense Plate	varchar(50)	
	Status	varchar(50)	

5.tbItem

Key	Column Name	ប្រភេទទិន្នន័យ	Allow Null
PK	ItemID	int	
	Category	varchar(50)	
	Description	varchar(50)	
	Weight	varchar(10)	
FK	CustomerID	int	
	CustomerName	nvarchar(50)	
	CustomerContact	varchar(20)	
FK	TransportID	int	
	TransportDate	date	
	Status	varchar(50)	

6.tbBooking

Key	Column Name	ប្រភេទទិន្នន័យ	Allow Null
PK	BookingID	int	
	Date	date	
	TotalPaid	money	
FK	CustomerID	int	
	CustomerName	varchar(50)	
	ContactInfo	varchar(20)	
FK	StaffID	int	
	StaffPosition	varchar(100)	
FK	VehicleID	int	
	Type	varchar(50)	
	SeatNum	tinyint	
FK	TargID	int	
	Destination	varchar(50)	

7.tbPayment

Key	Column Name	ប្រភេទទិន្នន័យ	Allow Null
PK	PaymentID	int	
	PayDate	date	
	PaidAmount	money	
FK	CustomerID	int	
	CustomeName	varchar(50)	
	CustomerContactInfo	varchar(20)	
FK	StaffID	int	
	StaffName	nvarchar(50)	
	StaffPosition	varchar(100)	

8.Transport

Key	Column Name	ប្រភេទទិន្នន័យ	Allow Null
PK	TransportID	int	
	TransportDate	date	
	Status	varchar(50)	
FK	CustomerID	int	
FK	StaffID	int	
	StaffPosition	varchar(100)	
FK	TargetID	int	
	Destination	varchar(50)	

9.tbInvoice

Key	Column Name	ប្រភេទទិន្នន័យ	Allow Null
	InvoiceID	int	
	InvoiceDate	date	
	TotalAmount	money	
	PaidAmount	money	
FK	CustomerID	int	
	CustomeName	varchar(50)	
	CustomerContactInfo	varchar(20)	
FK	StaffID	int	
	StaffName	nvarchar(50)	
	StaffPosition	varchar(100)	

10. Ticket

Key	Column Name	ប្រភេទទិន្នន័យ	Allow Null
PK	TicketID	int	
	SeatNum	tinyint	
	Date	date	
	Price	varchar(20)	
FK	CustomerID	int	
	ContactInfo	varchar(20)	
FK	StaffID	int	
	StaffPosition	varchar(100)	
FK	BookingID	int	
	BookingDate	date	

11.tbUser

Key	Column Name	ប្រភេទទិន្នន័យ	Allow Null
PK	UserID	int	
	Username	varchar(20)	
	Password	varchar(20)	
FK	StaffID	int	
	StaffName	varchar(50)	
	StaffPosition	varchar(100)	

II. ការរចនា User Interface Layout

User Interface គឺជាផ្ទៃសំរាប់ទំនាក់ទំនងជាមួយនឹងអ្នកប្រើប្រាស់ដើម្បីឲ្យអ្នកប្រើប្រាស់អាចដំណើរការទៅលើប្រព័ន្ធព័ត៌មានដូចជាបញ្ចូលទិន្នន័យ ស្វែងរកព័ត៌មានរួចកែប្រែឬលុបព័ត៌មានដែលរកឃើញជាដើម ។

1.Customer's Form

Customer's Information

— □ ×

Search :

Customer ID :

Customer Name :

Customer Age :

Customer Gender :

Customer Contact :

Insert

Update

Log Out

Back

Log Out

2.Staff's Form

Staff's Information

Search :

Staff ID :

First Name :

Last Name :

Gender :

Birth Date :

Position :

House No :

Street No :

Sangkat :

Khann :

Province/ City :

Contact Number :

Personal Number :

Salary :

Hire Date:

☐ Stop Work

Insert

Update

New

Back

Log Out

3.Route' Form

Route's Information — □ ×

Route ID :
Origin :
Destination:
Distance :
Search By : ▼

RouteID	Origin	Destination	Destination

4. Vehicle Form

Vehicle's Information — □ ×

Vehicle ID :
Type :
Capacity:
License Plate :
Status : ▼
Search By : ▼

VehicleID	Type	Capacity	License Plat	Status

5. Item's Form

Item's Information

Item ID :

Category :

Description :

Weight :

Customer ID :

Customer Name :

Customer Contact :

Transport ID :

Transport Date :

Status :

Insert

Update

New

Log Out

6. Booking's Form

Booking's Information

Booking ID :

Booking Date :

Total Prepaid:

Status :

Customer ID :

Customer Name :

Customer Contact :

Staff ID :

Staff Position :

Vehicle ID :

Seat Number :

Route ID :

Destination :

Search By :

Insert

Update

New

Back

Log Out

7. Payment's Form

Payment's Information

Search By :

Payment ID :

Pay Date :

Paid Amount :

Customer ID :

Customer Name :

Customer Contact :

Staff ID :

Staff Name :

Staff Position :

8. Transport's Form

Transport's Information

Transport ID :

Transport Date :

Status :

Customer ID :

Search By :

Staff ID :

Staff Position :

Route ID :

Destination :

9. Invoice's Form

Invoice's Information

Search By :

Invoice ID :

Invoice Date :

Total Amount :

Paid Amount :

Customer ID :

Customer Name :

Customer Contact :

Staff ID :

Staff Name :

Staff Position :

Insert

Update

New

Log Out

10. Ticket's Form

Ticket's Information

Ticket ID :

Customer ID :

Seat Number:

Customer Contact :

Date :

Staff ID :

Price:

Staff Position :

Search By :

Insert

Update

New

Back

Log Out

11. User's Form

User's Information

— □ ×

Search :

User ID :

Username :

Password :

Staff ID :

Staff Name :

Staff Position :

Insert

Update

New

Log Out

ដំណាក់កាលទីIV

ការដំឡើងប្រព័ន្ធ (System implementation)

ដំណាក់កាលទី ៤ នេះជាដំណាក់កាលដំឡើងប្រព័ន្ធ (System implementation) មានការចូលរួមពីប្រព័ន្ធដោយមានសកម្មភាពដូចជា៖

- បង្កើត Database រួចបង្កើត table in DBMS
- យក Application ឬ Programming Language ដែលអ្នក វិភាគបានកំណត់ មកបង្កើត User Interfaces រួចសរសេរកូដប្រកាសសំរាប់ Interface នីមួយៗ
- ធ្វើតេស្តសាកល្បងរួចដាក់បញ្ចូលគ្នារវាង program ទាំងនោះ
- ធ្វើឯកសារបង្រៀនអ្នកប្រើប្រាស់អោយចេះប្រើ User Interface ទាំងនោះ។

I. ការដំឡើងនៅផ្នែក Server (Server-Side Implementation)

Microsoft SQL Server ត្រូវបានជ្រើសរើសដើម្បីធ្វើជា DBMS ទុក សម្រាប់ផ្ទុកទិន្នន័យ។ ហើយយើងនឹងប្រើប្រាស់ Script File មួយដើម្បីបង្កើត database, tables និង stored procedures ទាំងអស់សម្រាប់ប្រើប្រាស់ ជាមួយនឹង database។ ដើម្បីយល់ច្បាស់បន្ថែមទៀត សូមអាន Appendix C: Database Programming ។

II. ការដំឡើងនៅផ្នែក client (Client-side Implementation)

នៅក្នុងការដំឡើងនៅផ្នែក Client យើងបានជ្រើសរើសយក ADO.NET ដើម្បីជាមធ្យោបាយក្នុងការតភ្ជាប់និងប្រើប្រាស់ Database ដូចជាការបញ្ចូលកែប្រែ ឬ ស្វែងរកទិន្នន័យជាដើម។

III, ការធ្វើឯកសារបច្ចេកទេសអ្នកប្រើប្រាស់ (User Interface)

1, Customer's Form

Customer's Form គឺជា user interface សំរាប់ធ្វើការជាមួយនឹង Customer ដែលក្នុងនោះ User អាចបញ្ចូលកែប្រែនិងស្វែងរក ទិន្នន័យបាន។

Customer's Form

Customer's Form

Copyright ISAD_M3_Group5(2024)

Search By :

5. momo
1001. van
1002. Hing Chab
1003. popa
1004. Kok Coca
1005. kkkkkkk
1006. ttttyt
1007. III
1008. unu

Customer ID : 1002

Customer Name : Hing Chab

Customer Age: 34

Customer Gender : Male

Customer ContactInfo: 0987654434

Insert Update Clear Back Log Out

ផ្ទៀងផ្ទាត់ភាពត្រឹមត្រូវនៃទិន្នន័យ (Data Validation)

- ចំពោះ CustomerID គឺជា field ដែលមានលក្ខណៈ auto-generated ។
- ចំពោះ Name គឺជា fields ដែលអាចបញ្ចូលបានតែ ៥០ តួអក្សរ បុណ្ណោះ។
- ចំពោះ Age គឺជា fields ដែលសម្រាប់ដឹងអំពីតួលេខនៃអាយុ។
- ចំពោះ Gender គឺជា fields ដែលអាចបញ្ចូលបានតែ ៧ តួអក្សរ បុណ្ណោះ។
- ចំពោះ ContactInfo គឺជា fields ដែលអាចដឹងពីព័ត៌មាន អាចបញ្ចូលបាន ២០ តួ។

ដំណើរការនៅក្នុង Form (Operations)

- ❖ Button Insert មាននាទីបញ្ចូលទិន្នន័យ Customer មួយ Record ចូលក្នុង Database ។
- ❖ ដើម្បី Search អ្នកប្រើប្រាស់ត្រូវប្រើទិន្នន័យដែលចង់វាយបញ្ចូល ហើយស្វែងរកនៅក្នុង Field search (CustomerID) រួចទិន្នន័យ Customer នឹងបង្ហាញក្នុង List Box ។ បន្ទាប់មកអ្នកប្រើប្រាស់អាច search រកទិន្នន័យនិងបានហើយ។

- ❖ Button Update មានតួនាទីកែប្រែទិន្នន័យ Customer ដែលបានជ្រើសរើសក្នុង List Box in Database ។
- ❖ Button Clear មាននាទីសំអាត Control ទាំងអស់នៅក្នុង Form ដើម្បីអោយអ្នកប្រើប្រាស់បញ្ចូលទិន្នន័យថ្មី។
- ❖ Button Back មាននាទីបិទ Form រួចត្រឡប់ទៅ Main Form ។
- ❖ Button Log Out មាននាទីត្រឡប់ទៅកាន់ Login Form វិញដោយមិនត្រូវបានឆ្លងកាត់ Main Form ។

2. Staff's Form

រង្វង់ផ្គត់ផ្គង់ព័ត៌មានត្រឹមត្រូវនៃទិន្នន័យ (Data Validation)

- ចំពោះ StaffID គឺជា field ដែលមានលក្ខណៈ auto-generated ។
- ចំពោះ First Name និង Last Name គឺជា fields ដែលអាចបញ្ចូលបានតែ 50 តួអក្សរប៉ុណ្ណោះ។
- ចំពោះ Gender គឺជា fields ដែលអាចបញ្ចូលបានតែ 7 តួអក្សរប៉ុណ្ណោះ។
- ចំពោះ BirthDate ត្រូវជ្រើសរើសកាលបរិច្ឆេទពីមុន។
- ចំពោះ Position គឺជា fields ដែលអាចបញ្ចូលបានតែ 100 តួអក្សរប៉ុណ្ណោះ ឬជ្រើសរើសទិន្នន័យដែលមានស្រាប់។

- ចំពោះ HouseNo គឺជា fields ដែលអាចបញ្ចូលបានតែ 15 តួអក្សរបុណ្ណោះ។
- ចំពោះ StreetNo គឺជា fields ដែលអាចបញ្ចូលបានតែ 25 តួអក្សរបុណ្ណោះ។
- ចំពោះ Sangkat, Khann,Province/City គឺត្រូវជ្រើសរើសតាមលំដាប់លំដោយ ដើម្បីអោយទិន្នន័យបង្ហាញមក combo box នីមួយៗមានលក្ខណៈត្រឹមត្រូវ។
- ចំពោះ ContactNumber និង PersonalNumberគឺជា field ដែលអាចវាយបញ្ចូលបានតាម format របស់ text boxes។
- ចំពោះ Salary គឺជា field ដែលអាចវាយបញ្ចូលបានតែចំនួនគត់ឬទសភាគបុណ្ណោះ ដែលស្ថិតនៅចន្លោះ១ដល់១០០០០០០០ ។

ដំណើរការនៅក្នុង Form(Operations)

- ❖ Button Insert មាននាទីបញ្ចូលទិន្នន័យ Staff មួយ Record ចូលក្នុង Database។
- ❖ ដើម្បី Search អ្នកប្រើប្រាស់ត្រូវប្រើទិន្នន័យដែលចង់វាយបញ្ចូល ហើយស្វែងរកនៅក្នុង Field search(StaffID)រួចទិន្នន័យ Staff និងបង្ហាញក្នុង List Box។ បន្ទាប់មកអ្នកប្រើប្រាស់អាច search រកទិន្នន័យនិងបានហើយ។
- ❖ Button Update មានតួនាទីកែប្រែទិន្នន័យ Staff ដែលបានជ្រើសរើសក្នុង List Box in Database ។
- ❖ Button Clear មាននាទីសំអាត Control ទាំងអស់នៅក្នុង Form ដើម្បីអោយអ្នកប្រើប្រាស់បញ្ចូលទិន្នន័យថ្មី។
- ❖ Button Back មាននាទីបិទ Form រួចត្រឡប់ទៅ Main Form។
- ❖ Button Log Out មាននាទីត្រឡប់ទៅកាន់ Login Form វិញដោយមិនត្រូវបានឆ្លងកាត់ Main Form ។

3,Route's Form

Route's Form

Copyright ISAD_M3_Group5(2024)

BUS TRAVEL

Route ID : 1004

Origin :

Destination :

Distance :

Search ID:

Insert Update Clear Back Log Out

	RouteID	Origin	Destination	Distance	
▶	3	pp	kandal	123km	
	1001	pp	kpc	120km	
	1002	pp	kps	100km	
	1003	Phnom Pen	Kratie	120km	

រឿងផ្ដាត់ភាពត្រឹមត្រូវនៃទិន្នន័យ (Data Validation)

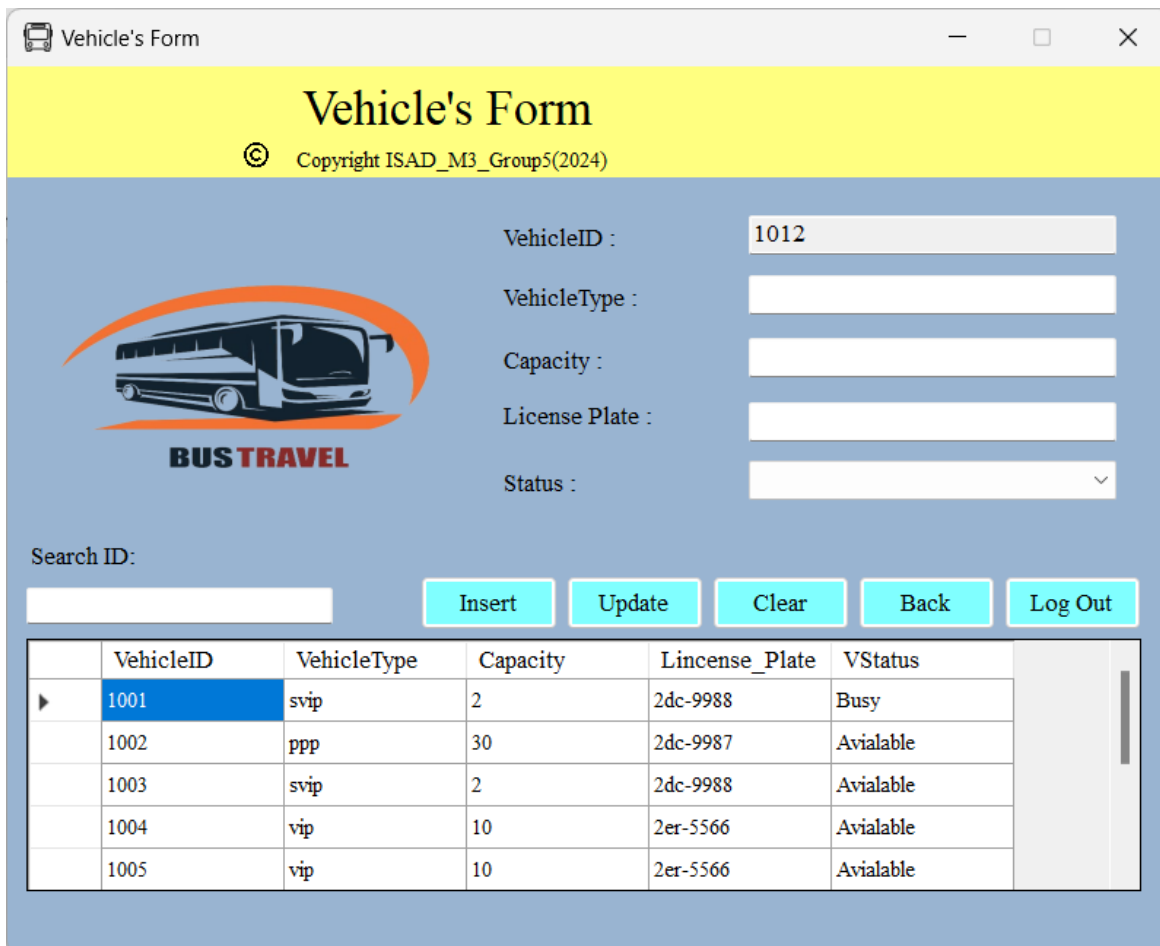
- ចំពោះ RouteID គឺជា field ដែលមានលក្ខណៈ auto-generated។
- ចំពោះ Origin គឺជា fields ដែលអាចបញ្ចូលបានតែ ៥០តួអក្សរអូរ៉ាស៊ី។
- ចំពោះ Destination fields ដែលអាចបញ្ចូលបានតែ ៥០តួអក្សរអូរ៉ាស៊ី។
- ចំពោះ Distance គឺជា fields ដែលអាចបញ្ចូលបានតែ ៥០តួអក្សរអូរ៉ាស៊ី។

ដំណើរការនៅក្នុង Form (Operations)

- ❖ Button Insert មាននាទីបញ្ចូលទិន្នន័យ Route មួយ Record ចូលក្នុង Database។
- ❖ ដើម្បី Search អ្នកប្រើប្រាស់ត្រូវប្រើទិន្នន័យដែលចង់វាយបញ្ចូល ហើយស្វែងរកនៅក្នុង Field search(RouteID)រួចទិន្នន័យ Routeនឹងបង្ហាញក្នុង Data Grid View។ បន្ទាប់មកអ្នកប្រើប្រាស់អាច Search រកទិន្នន័យនិងបានហើយ។
- ❖ Button Update មានតួនាទីកែប្រែទិន្នន័យ Routeដែលបានជ្រើសរើសក្នុង Data Grid View in Database។
- ❖ Button Clear មាននាទីសំអាត Control ទាំងអស់នៅក្នុង Form ដើម្បីអោយអ្នកប្រើប្រាស់បញ្ចូលទិន្នន័យថ្មី។

- ❖ Button Back មាននាទីបិទ Form រួចត្រឡប់ទៅ Main Form ។
- ❖ Button Log Out មាននាទីត្រឡប់ទៅកាន់ Login Form វិញដោយមិនត្រូវបានឆ្លងកាត់ Main Form ។
- ❖ Button Clear ត្រូវបានគេប្រើសម្រាប់ជម្រះ ទិន្នន័យមុនចេញពី Data Grid View ។

4. Vehicle's Form



Vehicle's Form

Copyright ISAD_M3_Group5(2024)

VehicleID : 1012

VehicleType :

Capacity :

License Plate :

Status :

Search ID:

Insert Update Clear Back Log Out

VehicleID	VehicleType	Capacity	Lincense_Plate	VStatus
1001	svip	2	2dc-9988	Busy
1002	ppp	30	2dc-9987	Avialable
1003	svip	2	2dc-9988	Avialable
1004	vip	10	2er-5566	Avialable
1005	vip	10	2er-5566	Avialable

រង្វង់ផ្គត់ផ្គង់ព័ត៌មានត្រឹមត្រូវនៃទិន្នន័យ (Data Validation)

- ចំពោះ VehicleID គឺជា field ដែលមានលក្ខណៈ auto-generated ។
- ចំពោះ Type គឺជា fields ដែលអាចបញ្ចូលបានតែ ៥០ តួអក្សរ ឬល្អោះ។
- ចំពោះ Capacity គឺជា fields ដែលអាចបញ្ចូលបានតែ ៥០ តួអក្សរ ឬល្អោះ។
- ចំពោះ Lincense Plate គឺជា fields ដែលអាចបញ្ចូលបានតែ ៥០ តួអក្សរ ឬល្អោះ។
- ចំពោះ Status គឺជា fields ដែលអាចបញ្ចូលបានតែ ៥០ តួអក្សរ ឬល្អោះ។

ដំណើរការនៅក្នុង Form(Operations)

- ❖ Button Insert មាននាទីបញ្ចូលទិន្នន័យ Vehicle មួយ Record ចូលក្នុង Database ។
- ❖ ដើម្បី Search អ្នកប្រើប្រាស់ត្រូវប្រើទិន្នន័យដែលចង់វាយបញ្ចូល ហើយស្វែងរកនៅក្នុង Field search(VehicleID)រួចទិន្នន័យ Vehicleនិងបង្ហាញក្នុង Data Grid View ។ បន្ទាប់មកអ្នកប្រើប្រាស់អាច Search រកទិន្នន័យនិងបានហើយ។
- ❖ Button Update មានតួនាទីកែប្រែទិន្នន័យ Vehicleដែលបានជ្រើសរើសក្នុង Data Grid View in Database ។
- ❖ Button Back មាននាទីបិទ Form រួចត្រឡប់ទៅ Main Form ។
- ❖ Button Log Out មាននាទីត្រឡប់ទៅកាន់ Login Form វិញដោយមិនត្រូវបានឆ្លងកាត់ Main Form ។
- ❖ Button Clear ត្រូវបានគេប្រើវាសម្រាប់ជម្រះទិន្នន័យមុនចេញពី Controls ។

5. Item's Form

ItemID	Category	CustomerID	CustomerName	TransportID	TransportDate
4	ppppp	1001	van	102	6/11/2024
5	sdgf	5	momo	6	6/13/2024
6	adf	5	momo	6	7/3/2024
7	gsdfg	5	momo	102	6/10/2024
101	Computer	1002	Hing Chab	103	6/10/2024

រៀបចំការត្រួតពិនិត្យទិន្នន័យ (Data Validation)

- ចំពោះ ItemID គឺជា field ដែលមានលក្ខណៈ auto-generated ។
- ចំពោះ Category គឺជា fields ដែលអាចបញ្ចូលបានតែ 50 តួអក្សរប៉ុណ្ណោះ។
- ចំពោះ Description គឺជា fields ដែលអាចបញ្ចូលបានតែ 50 តួអក្សរប៉ុណ្ណោះ។
- ចំពោះ Weight គឺជា fields ដែលអាចបញ្ចូលបានតែ 10 តួអក្សរប៉ុណ្ណោះ។
- ចំពោះ TransportID គឺត្រូវជ្រើសរើសតាមលំដាប់លំដោយ ដើម្បីអោយទិន្នន័យបង្ហាញមក combo box នីមួយៗមានលក្ខណៈត្រឹមត្រូវ។
- ចំពោះ TransportDate គឺជា fields ដែលអាចបញ្ចូលបានតែ 100 តួអក្សរប៉ុណ្ណោះ ឬជ្រើសរើសទិន្នន័យដែលមានស្រាប់។

ដំណើរការនៅក្នុង Form (Operations)

- ❖ Button Insert មាននាទីបញ្ចូលទិន្នន័យ Item មួយ Record ចូលក្នុង Database ។
- ❖ ដើម្បី Search អ្នកប្រើប្រាស់ត្រូវប្រើទិន្នន័យដែលចង់រាយបញ្ចូល ហើយស្វែងរកនៅក្នុង Field search(ItemID) រួចទិន្នន័យ Item នឹងបង្ហាញក្នុង Data Grid View ។ បន្ទាប់មកអ្នកប្រើប្រាស់អាច Search រកទិន្នន័យនិងបានហើយ។

- ❖ Button Update មានតួនាទីកែប្រែទិន្នន័យItemដែលបានជ្រើសរើសក្នុង Data Grid View in Database ។
- ❖ Button Clear មាននាទីសំអាត Control ទាំងអស់នៅក្នុង Form ដើម្បីអោយអ្នកប្រើប្រាស់បញ្ចូលទិន្នន័យថ្មី។
- ❖ Button Back មាននាទីធ្វើអោយត្រឡប់ទៅ Main Form ។
- ❖ Button Log Out មាននាទីត្រឡប់ទៅកាន់Login Form វិញដោយមិនត្រូវបានឆ្លងកាត់ ។

6. Booking's Form



Booking's Form

© Copyright ISAD_M3_Group5(2024)

BookingID :

BookingDate :

Total Paid :

Customer ID :

Customer Name :

Customer Contact :

Route ID :

Search BookingID :

Destination :

Vehicle ID :

Type :

Status :

Staff ID :

StaffNameKh :

Staff Position :

	BookingID	BookingDate	TotalPaid	CustomerID	RouteID	VehicleID	StaffID
▶	102	6/11/2024	9.0000	1002	1002	1001	1003
	103	6/28/2024	34.0000	1001	1002	1006	1002
	104	6/20/2024	15.0000	1008	1002	1006	1001
	105	6/21/2024	120.0000	1002	1002	1005	1003
	106	6/21/2024	134.0000	1001	1001	1003	7

រង្វង់ផ្គត់ផ្គង់ត្រឹមត្រូវនៃទិន្នន័យ (Data Validation)

- ចំពោះ BookingID គឺជា field ដែលមានលក្ខណៈ auto-generated ។
- ចំពោះ Booking Date ត្រូវជ្រើសរើសកាលបរិច្ឆេទពីមុន។
- ចំពោះ Total Paid គឺជា field ដែលអាចវាយបញ្ចូលបានតែចំនួនគត់ឬទសភាគបុណ្ណោះ ដែលស្ថិតនៅចន្លោះ ១ ដល់ ១០០០០០០០ ។

ដំណើរការនៅក្នុង Form (Operations)

- ❖ Button Insert មាននាទីបញ្ចូលទិន្នន័យ Booking មួយ Record ចូលក្នុង Database ។
- ❖ ដើម្បី Search អ្នកប្រើប្រាស់ត្រូវប្រើទិន្នន័យដែលចង់វាយបញ្ចូល ហើយស្វែងរកនៅក្នុង Field Search(BookingID) រួចទិន្នន័យ Booking នឹងបង្ហាញក្នុង Data Grid View បន្ទាប់មកអ្នកប្រើប្រាស់អាច search រកទិន្នន័យនិងបានហើយ។
- ❖ Button Update មានតួនាទីកែប្រែទិន្នន័យ Item ដែលបានជ្រើសរើសក្នុង Data Grid View in Database ។
- ❖ Button Back មាននាទីធ្វើអោយត្រឡប់ ទៅកាន់ Main Form ។
- ❖ Button Log Out មាននាទីត្រឡប់ទៅកាន់ Login Form វិញដោយមិនត្រូវបានឆ្លងកាត់ Main Form ។
- ❖ Button Clear ត្រូវបានគេប្រើសម្រាប់ជម្រះ ទិន្នន័យមុនចេញពី Controls ។

7. Payment's Form

រង្វង់ផ្គត់ផ្គង់ព័ត៌មានត្រឹមត្រូវនៃទិន្នន័យ (Data Validation)

- ចំពោះ PaymentID ជា field ដែលមានលក្ខណៈ auto-generated ។
- ចំពោះ Paid Date ត្រូវជ្រើសរើសកាលបរិច្ឆេទពីមុន ។
- ចំពោះ Paid Amount គឺជា field ដែលអាចវាយបញ្ចូលបានតែចំនួនគត់ឬទសភាគប៉ុណ្ណោះ ដែលស្ថិតនៅចន្លោះ ១ ដល់ ១០០០០០០ ។

ដំណើរការនៅក្នុង Form (Operations)

- ❖ Button Insert មាននាទីបញ្ចូលទិន្នន័យ Payment មួយ Record ចូលក្នុង Database ។
- ❖ ដើម្បី Search អ្នកប្រើប្រាស់ត្រូវប្រើទិន្នន័យដែលចង់វាយបញ្ចូល ហើយស្វែងរកនៅក្នុង Field search(PaymentID) រួចទិន្នន័យ Payment នឹងបង្ហាញក្នុង List Box ។ បន្ទាប់មកអ្នកប្រើប្រាស់អាច search រកទិន្នន័យនិងបានហើយ ។

- ❖ Button Update មានតួនាទីកែប្រែទិន្នន័យItemដែលបានជ្រើសរើសក្នុង List Box in Database ។
- ❖ Button Back មាននាទីធ្វើអោយត្រឡប់មកយក្រោយបន្ទាប់ពីមុន (២ត្រឡប់មក១វិញ) ។
- ❖ Button Log Out មាននាទីត្រឡប់ទៅកាន់Login Form វិញដោយមិនត្រូវបានឆ្លងកាត់Main Form ។
- ❖ Button Clearត្រូវបានគេប្រើសម្រាប់ជម្រះ ទិន្នន័យមុនចេញពី List box។

8. Transport's Form

TransportID	TransportDate	Status	CustomerID	CustomerInfo	RouteID	Destination
102	6/10/2024	Start	1001	0123456789	1001	kpc
103	6/10/2024	Start	1001	0123456789	1001	kpc
104	6/20/2024	Start	1002	0987654434	1001	kpc
105	6/20/2024	On go in	1003	09876554332	1001	kpc

ផ្ទៀងផ្ទាត់ភាពត្រឹមត្រូវនៃទិន្នន័យ (Data Validation)

- ចំពោះ TransportID ជា field ដែលមានលក្ខណៈ auto-generated ។
- ចំពោះ Transport Dateត្រូវជ្រើសរើសកាលបរិច្ឆេទពីមុន ។
- CustomerID System និងទាញយកតាម combobox រួចផ្ទៀងផ្ទាត់ឈ្មោះរបស់អតិថិជនដែលបានបង្ហាញនៅក្នុង CustomerName ។

ដំណើរការនៅក្នុងForm (Operations)

- ❖ Button Insert មាននាទីបញ្ចូលទិន្នន័យ Transport មួយ Recordចូលក្នុង Database ។
- ❖ ដើម្បី Search អ្នកប្រើប្រាស់ត្រូវប្រើទិន្នន័យដែលចង់រាយបញ្ចូល ហើយស្វែងរកនៅក្នុង Field search(TransportID)រួចទិន្នន័យ Transportនិងបង្ហាញក្នុង Data Grid View ។ បន្ទាប់មកអ្នកប្រើប្រាស់អាច search រកទិន្នន័យនិងបានហើយ។

- ❖ Button Update មានតួនាទីកែប្រែទិន្នន័យItemដែលបានជ្រើសរើសក្នុង Data Grid View in Database ។
- ❖ Button Back មាននាទីធ្វើអោយត្រឡប់ ទៅកាន់ Main Form ។
- ❖ Button Log Out មាននាទីត្រឡប់ទៅកាន់ Login Form វិញដោយមិនត្រូវបានឆ្លងកាត់Main Form ។
- ❖ Button Clear មាននាទីសំអាត Control ទាំងអស់នៅក្នុង Form ដើម្បីអោយអ្នកប្រើប្រាស់បញ្ចូលទិន្នន័យថ្មី។

9. Invoice's Form

ផ្ទៀងផ្ទាត់ភាពត្រឹមត្រូវនៃទិន្នន័យ (Data Validation)

- ចំពោះ InvoiceID ជា field ដែលមានលក្ខណៈ auto-generated ។
- ចំពោះ Invoice Date ត្រូវជ្រើសរើសកាលបរិច្ឆេទពីមុន។
- ចំពោះ Total Amount ជាតម្លៃដែលទាញមកពី CustomerID ចំនួនសាច់ប្រាក់។
- ចំពោះ Paid Amount គឺជា field ដែលអាចវាយបញ្ចូលបានតែចំនួនគត់ឬទសភាគបុណ្ណោះ ដែលស្ថិតនៅចន្លោះ១ដល់១០០០០០០០ ។
- CustomerID ប្រើក្នុង Combo box ។

- CustomerID System និងទាញយកតាម contactInfo រួចផ្ទៀងផ្ទាត់ឈ្មោះរបស់អតិថិជនដែលបានបង្ហាញនៅក្នុង CustomerName ។
- StaffID System និងទាញយកcombobox រួចផ្ទៀងផ្ទាត់ឈ្មោះរបស់បុគ្គលិក Staff Name and Staff Positionដែលបានបង្ហាញនៅក្នុង textbox ។

ដំណើរការនៅក្នុងForm(Operations)

- ❖ Button Insert មាននាទីបញ្ចូលទិន្នន័យ Invoiceមួយ Record ចូលក្នុង Database ។
- ❖ ដើម្បី Search អ្នកប្រើប្រាស់ត្រូវប្រើទិន្នន័យដែលចង់វាយបញ្ចូល ហើយស្វែងរកនៅក្នុង Field search(InvoiceID)រួចទិន្នន័យ Invoiceនិងបង្ហាញក្នុង List Box ។ បន្ទាប់មកអ្នកប្រើប្រាស់អាច search រកទិន្នន័យនិងបានហើយ។
- ❖ Button Update មាននាទីកែប្រែទិន្នន័យItemដែលបានជ្រើសរើសក្នុង List Box in Database ។
- ❖ Button Back មាននាទីធ្វើអោយត្រឡប់ ទៅកាន់ Main Form ។
- ❖ Button Log Out មាននាទីត្រឡប់ទៅកាន់Login Form វិញដោយមិនត្រូវបានឆ្លងកាត់Main Form ។
- ❖ Button Clearត្រូវបានគេប្រើសម្រាប់ជម្រះ ទិន្នន័យមុនចេញពី List box។

10, Ticket's Form


Ticket's Form



Ticket's Form

© Copyright ISAD_M3_Group5(Roza)



TicketID :

1

Seat Number :

5ac

Price :

1999.0000

Ticket Date :

06/21/2024

Customer ID :

1004

Customer Contact :

012345678

Staff ID :

1002

Staff Position :

Cashier

Search TicketID :

Insert

Update

Clear

Back

Log Out

	TicketID	SeatNumber	Price	TickeDate	CustomerID	CustomerContac
▶	1	5ac	1999.0000	6/21/2024	1004	012345678
	2	3a	16.0000	6/21/2024	1002	0987654434
	3	3e	12.0000	6/21/2024	1001	0123456789

ផ្ទៀងផ្ទាត់ភាពត្រឹមត្រូវនៃទិន្នន័យ (Data Validation)

- ចំពោះ TicketID ជា field ដែលមានលក្ខណៈ auto-generated ។
- ចំពោះ Seat Number គឺជា fields ដែលអាចបញ្ចូលបានតែ 20 តួអក្សររហូតប៉ុណ្ណោះ។
- ចំពោះ Ticket Date ត្រូវជ្រើសរើសកាលបរិច្ឆេទពីមុន។
- ចំពោះ Price តម្លៃរបស់សំបុត្រ។
- CustomerID ប្រើក្នុង Combo box ។
- CustomerID System និងទាញយកតាម combobox រួចផ្ទៀងផ្ទាត់ឈ្មោះរបស់អតិថិជនដែលបានបង្ហាញនៅក្នុង textbox ។
- StaffID System និងទាញយក combobox រួចផ្ទៀងផ្ទាត់ឈ្មោះរបស់បុគ្គលិក Staff Name and Staff Position ដែលបានបង្ហាញនៅក្នុង textbox ។

ដំណើរការនៅក្នុង Form (Operations)

- Button Insert មាននាទីបញ្ចូលទិន្នន័យ Invoice មួយ Record ចូលក្នុង Database ។
- ដើម្បី Search អ្នកប្រើប្រាស់ត្រូវប្រើទិន្នន័យដែលចង់វាយបញ្ចូល ហើយស្វែងរកនៅក្នុង Field search (TicketID) រួចទិន្នន័យ Ticket និងបង្ហាញក្នុង Data Grid View ។ បន្ទាប់មកអ្នកប្រើប្រាស់អាច Search រកទិន្នន័យនិងបានហើយ។
- Button Update មាននាទីកែប្រែទិន្នន័យ Item ដែលបានជ្រើសរើសក្នុង List Box in Database ។
- Button Back មាននាទីធ្វើអោយត្រឡប់ ទៅកាន់ Main Form ។
- Button Log Out មាននាទីត្រឡប់ទៅកាន់ Login Form វិញដោយមិនត្រូវបានឆ្លងកាត់ Main Form ។
- Button Clear មាននាទីសំអាត Control ទាំងអស់នៅក្នុង Form ដើម្បីអោយអ្នកប្រើប្រាស់បញ្ចូលទិន្នន័យថ្មី។

11. User's Form

រៀបចំផ្ទាំងតាមត្រឹមត្រូវនៃទិន្នន័យ (Data Validation)

- ចំពោះ UserID ជា field ដែលមានលក្ខណៈ auto-generated ។
- ចំពោះ Username គឺជា fields ដែលអាចបញ្ចូលបានតែ ៥០ តួអក្សរ ហើយអាចវាយបានតែ English និង លេខប៉ុណ្ណោះ។
- ចំពោះ Password គឺជា fields ដែលអាចបញ្ចូលបានតែ ៥០ តួអក្សរ ហើយអាចវាយបានតែ English និង លេខប៉ុណ្ណោះ។
- StaffID System នឹងទាញយក combobox រួចផ្ទៀងផ្ទាត់ឈ្មោះរបស់បុគ្គលិក Staff Name and Staff Position ដែលបានបង្ហាញនៅក្នុង textbox ។

ដំណើរការនៅក្នុង Form (Operations)

- ❖ Button Insert មាននាទីបញ្ចូលទិន្នន័យ User មួយ Record ចូលក្នុង Database ។
- ❖ ដើម្បី Search អ្នកប្រើប្រាស់ត្រូវប្រើទិន្នន័យដែលចង់វាយបញ្ចូល ហើយស្វែងរកនៅក្នុង Field Search (UserID) រួចទិន្នន័យ User នឹងបង្ហាញក្នុង List Box ។ បន្ទាប់មកអ្នកប្រើប្រាស់អាច search រកទិន្នន័យនិងបានហើយ។
- ❖ Button Update មានតួនាទីកែប្រែទិន្នន័យ User ដែលបានជ្រើសរើសក្នុង List Box in Database ។
- ❖ Button Back មាននាទីធ្វើអោយត្រឡប់ថយទៅ Main Form ។
- ❖ Button Log Out មាននាទីត្រឡប់ទៅកាន់ Login Form វិញដោយមិនត្រូវបានឆ្លងកាត់ Main Form ។
- ❖ Button Clear ត្រូវបានគេប្រើសម្រាប់ជម្រះ ទិន្នន័យមុនចេញពី List box ។

Code (Implementing Programming Language)

ដើម្បីផ្ទុក និងទាញទិន្នន័យពី Database Classes យើងមាន ៖

I. Classes

```

sing System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace PAGTMS.Models;

public class Staff
{
    public int StaffID { get; set; }
    public string? StaffNameKH { get; set; }
    public string? StaffNameEN { get; set; }
    public string? Gender { get; set; }
    public DateTime BirthDate { get; set; }
    public string? StaffPosition { get; set; }
    public string? HouseNo { get; set; }
    public string? StreetNo { get; set; }
    public string? Sangkat { get; set; }
    public string? Khann { get; set; }
    public string? Province { get; set; }
    public string? ContactNumber { get; set; }
    public string? PersonalNumber { get; set; } = "";
    public decimal? Salary { get; set; }
    public DateTime HiredDate { get; set; }
    public byte[]? Photo { get; set; } = null;
    public bool StopWork { get; set; }
}

```

II. IDataRecordExtension

```

public static class IDataRecordExtension
{
    public static Staff ToStaff(this IDataReader record)
    {
        int index = record.GetOrdinal("StaffID");
        int id = record.GetInt32(index);

        index = record.GetOrdinal("StaffNameKH");
        string? staffnameKH = record.GetString(index);
        return new Staff()
        {
            StaffID = id,
            StaffNameKH = staffnameKH,
        };
    }
    public static Staff ToStaffAllData(this IDataReader record)
    {
        int index = record.GetOrdinal("StaffID");
        int id = record.GetInt32(index);
    }
}

```

```

index = record.GetOrdinal("StaffNameKH");
string? staffnameKH = record.GetString(index);

index = record.GetOrdinal("StaffNameEN");
string? staffnameEN = record.GetString(index);
index = record.GetOrdinal("Gender");
string? gender = record.GetString(index);

index = record.GetOrdinal("BirthDate");
DateTime birthdate = record.GetDateTime(index);

index = record.GetOrdinal("StaffPosition");
string? staffposition = record.GetString(index);

index = record.GetOrdinal("HouseNo");
string? houseno = record.GetString(index);

index = record.GetOrdinal("StreetNo");
string? streetno = record.GetString(index);

index = record.GetOrdinal("Khann");
string? Khann = record.GetString(index);

index = record.GetOrdinal("Sangkat");
string? sangkat = record.GetString(index);

index = record.GetOrdinal("Province");
string? province = record.GetString(index);

index = record.GetOrdinal("ContactNumber");
string? contactnumber = record.GetString(index);

index = record.GetOrdinal("PersonalNumber");
string? personalnumber = record.GetString(index);
if (!record.IsDBNull(index)) personalnumber = record.GetString(index);

index = record.GetOrdinal("Salary");
decimal? salary = record.GetDecimal(index);

index = record.GetOrdinal("HiredDate");
DateTime hireddate = record.GetDateTime(index);

index = record.GetOrdinal("Photo");
byte[]? photo = null;
if (!record.IsDBNull(index)) photo = (byte[])record[index];
index = record.GetOrdinal("StopWork");
bool stopwork = record.GetBoolean(index);

```

```

return new Staff()
{
    StaffID = id,
    StaffNameKH = staffnameKH,
    StaffNameEN = staffnameEN,
    Gender = gender,
    BirthDate = birthdate,
    StaffPosition = staffposition,
    HouseNo = houseno,
    StreetNo = streetno,
    Sangkat = sangkat,
    Khann = khann,
    Province = province,
    ContactNumber = contactnumber,
    PersonalNumber = personalnumber,
    Salary = salary,
    HiredDate = hireddate,
    Photo = photo,
    StopWork = stopwork,
};

}

}

```

III. Function Classes

```

using Microsoft.Data.SqlClient;
using System.Data;
using PAGTMS.Models;
namespace PAGTMS.Utills;

public static class StaffFunc
{
    public static IEnumerable<Staff> GetAllStaff(SqlConnection con)
    {
        SqlCommand cmd = new SqlCommand("spReadALLStaff", con);
        SqlDataReader? reader = null;
        try
        {
            reader = cmd.ExecuteReader();
        }
        catch (Exception ex)
        {
            throw new Exception($"Error in getting all Staff > {ex.Message}");
        }
        finally
        {
            cmd.Dispose();
        }
        if (reader != null && reader.HasRows == true)
        {
            var queryAbles = reader.Cast<IDataRecord>().AsQueryable();
            foreach (var record in queryAbles)
            {
                yield return reader.ToStaff();
            }
        }
        reader?.Close();
    }
}

```

```

public static Staff GetOneStaff(SqlConnection con, int id)
{
    SqlCommand cmd = new SqlCommand("spReadOneStaff", con);
    cmd.CommandType = CommandType.StoredProcedure;
    cmd.Parameters.AddWithValue("@sid", id);
    SqlDataReader? reader = null;
    try
    {
        reader = cmd.ExecuteReader();
    }
    catch (Exception ex)
    {
        throw new Exception($"Error in getting staff with id, {id} > {ex.Message}");
    }
    finally
    {
        cmd.Dispose();
    }
    Staff? result = null;
    if (reader != null && reader.HasRows == true)
    {
        if (reader.Read() == true)
        {
            result = reader.ToStaffAllData();
        }
    }
    reader?.Close();
    return result;
}

public static bool AddStaff(SqlConnection con, Staff staff)
{
    SqlCommand cmd = new SqlCommand("spInsertStaff", con);
    cmd.CommandType = CommandType.StoredProcedure;
    cmd.Parameters.AddWithValue("@sid", staff.StaffID);
    cmd.Parameters.AddWithValue("@snKH", staff.StaffNameKH);
    cmd.Parameters.AddWithValue("@snEN", staff.StaffNameEN);
    cmd.Parameters.AddWithValue("@gen", staff.Gender);
    cmd.Parameters.AddWithValue("@bd", staff.BirthDate);
    cmd.Parameters.AddWithValue("@pos", staff.StaffPosition);
    cmd.Parameters.AddWithValue("@sn", staff.StreetNo);
    cmd.Parameters.AddWithValue("@hn", staff.HouseNo);
    cmd.Parameters.AddWithValue("@khn", staff.Khann);
    cmd.Parameters.AddWithValue("@sk", staff.Sangkat);
    cmd.Parameters.AddWithValue("@pro", staff.Province);
    cmd.Parameters.AddWithValue("@cn", staff.ContactNumber);
    cmd.Parameters.AddWithValue("@pn", staff.PersonalNumber);
    cmd.Parameters.AddWithValue("@sa", staff.Salary);
    cmd.Parameters.AddWithValue("@hd", staff.HiredDate);
    cmd.Parameters.AddWithValue("@ph", staff.Photo);
    cmd.Parameters.AddWithValue("@spw", staff.StopWork);
    try
    {
        int effected = cmd.ExecuteNonQuery();
        return effected > 0;
        //return effected > 0 ? staff.StaffID : 0;
    }
    catch (Exception ex)
    {
        throw new Exception($"Failed in adding new staff > {ex.Message}");
    }
}

```

```

        finally
        {
            cmd.Dispose();
        }
    }
}

public static bool UpdateStaff(SqlConnection con, Staff staff)
{
    SqlCommand cmd = new SqlCommand("spUpdateStaff", con);
    cmd.CommandType = CommandType.StoredProcedure;
    cmd.Parameters.AddWithValue("@sid", staff.StaffID);
    cmd.Parameters.AddWithValue("@snKH", staff.StaffNameKH);
    cmd.Parameters.AddWithValue("@snEN", staff.StaffNameEN);
    cmd.Parameters.AddWithValue("@gen", staff.Gender);
    cmd.Parameters.AddWithValue("@bd", staff.BirthDate);
    cmd.Parameters.AddWithValue("@pos", staff.StaffPosition);
    cmd.Parameters.AddWithValue("@hn", staff.HouseNo);
    cmd.Parameters.AddWithValue("@sn", staff.StreetNo);
    cmd.Parameters.AddWithValue("@sk", staff.Sangkat);
    cmd.Parameters.AddWithValue("@khn", staff.Khann);
    cmd.Parameters.AddWithValue("@pro", staff.Province);
    cmd.Parameters.AddWithValue("@cn", staff.ContactNumber);
    cmd.Parameters.AddWithValue("@pn", staff.PersonalNumber);
    cmd.Parameters.AddWithValue("@sa", staff.Salary);
    cmd.Parameters.AddWithValue("@hd", staff.HiredDate);
    cmd.Parameters.AddWithValue("@ph", staff.Photo);
    cmd.Parameters.AddWithValue("@spw", staff.StopWork);
    try
    {
        int effected = cmd.ExecuteNonQuery();
        return (effected > 0);
    }
    catch (Exception ex)
    {
        throw new Exception($"Failed in updating existing staff > {ex.Message}");
    }
    finally
    {
        cmd.Dispose();
    }
}
}

```

Code (Implementing Windows Forms)

I. Model ទី 1 (ListBox control)

```

using PAGTMS.Models;
using PAGTMS.Utlils;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.Entity;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace PAGTMS;

public partial class CustomerForm : Form
{
    Customer? effectedCustomer = null;
    List<string> ListCustomer = new List<string>();
    int CustomerCount = 0;
    int IndexOfUpdateCustomer;

```

```

private MainForm mainFormReference;
public CustomerForm(MainForm mainform)
{
    InitializeComponent();
    mainFormReference = mainform;
    this.FormClosed += new FormClosedEventHandler(
mainFormReference.Show());
    btnInsert.Click += DoClickInsertCustomer;
    btnUpdate.Click += DoClickUpdateCustomer;
    btnClear.Click += DoClickClear;
    btnBack.Click += (_, _) => this.Close();
    LsCustomer.SelectedIndexChanged += Select_Handling_Customer;
    txtSearch.TextChanged += SearchFunction;
    ListCustomer.Clear();
    btnLogOut.Click += (_, _) => { this.Hide(); new LoginForm().Show(); };
}

private void SearchFunction(object? sender, EventArgs e)
{
    if (string.IsNullOrEmpty(txtSearch.Text) == false)
    {
        LsCustomer.Items.Clear();
        foreach (string customer in ListCustomer)
        {
            string[] splitText = customer.Split('.');
            if (splitText[1].Trim().StartsWith(txtSearch.Text,
StringComparison.OrdinalIgnoreCase))
            {
                LsCustomer.Items.Add(customer);
            }
        }
    }
    else if (txtSearch.Text == "")
    {
        LsCustomer.Items.Clear();
        foreach (string customer in ListCustomer)
        {
            LsCustomer.Items.Add(customer);
        }
    }
}

private void DoClickUpdateCustomer(object? sender, EventArgs e)
{
    if (effecteCustomer != null)
    {
        effecteCustomer.CustomerID = int.Parse(txtCid.Text.Trim());
        effecteCustomer.CustomerName = txtCusName.Text.Trim();
        effecteCustomer.Gender = cbSex.SelectedItem.ToString();
        effecteCustomer.CusAge = Int16.Parse(txtCusAge.Text);
        effecteCustomer.CustomerContactInfo = txtCusInfo.Text.Trim();

        try
        {
            var result = CustomerFunc.UpdateCustomer(Program.Connection,
effecteCustomer);
            if (result == true)
                MessageBox.Show($"Successfully updated an existing Customer with the
id : {effecteCustomer.CustomerID} Updated!", "Updated", MessageBoxButtons.OK,
MessageBoxIcon.Information);
            foreach (string checkcustomer in ListCustomer)
            {
                string[] splitText = checkcustomer.Split('.');
                if
(splitText[0].Trim().Equals(effecteCustomer.CustomerID.ToString()))

```

```

        {
            IndexOfUpdateCustomer = ListCustomer.IndexOf(checkcustomer);
            ListCustomer.Remove(checkcustomer)
            break;
        }
    }
    ListCustomer.Insert(IndexOfUpdateCustomer,
    $"{effectedCustomer.CustomerID}. {effectedCustomer.CustomerName}");
}
catch (Exception ex)
{
    MessageBox.Show($"Failed to update Updated! {ex.Message}", "Updating",
    MessageBoxButtons.OK, MessageBoxIcon.Error);
}
}
LoadingDataCustomer();
}
private void DoClickInsertCustomer(object? sender, EventArgs e)
{
    if (int.TryParse(txtCid.Text, out int customerID) == false)
    {
        MessageBox.Show("Customer ID is required", "Creating",
        MessageBoxButtons.OK, MessageBoxIcon.Stop);
        return;
    }
    if (txtCusName.Text == "")
    {
        MessageBox.Show("CustomerName is required", "Creating",
        MessageBoxButtons.OK, MessageBoxIcon.Stop);
        return;
    }
    if (cbSex.Text == "")
    {
        MessageBox.Show("Customer Gender is required", "Creating",
        MessageBoxButtons.OK, MessageBoxIcon.Stop);
        return;
    }
    if (Int16.TryParse(txtCusAge.Text, out Int16 cusAge) == false)
    {
        MessageBox.Show("Customer ID is required", "Creating",
        MessageBoxButtons.OK, MessageBoxIcon.Stop);
        return;
    }
    if (txtCusInfo.Text == "")
    {
        MessageBox.Show("Customer ContactInfo is required", "Creating",
        MessageBoxButtons.OK, MessageBoxIcon.Stop);
        return;
    }
    Customer newCustomer = new Customer()
    {
        CustomerID = customerID,
        CustomerName = txtCusName.Text.Trim(),
        Gender = cbSex.Text.Trim(),
        CusAge = cusAge,
        CustomerContactInfo = txtCusInfo.Text.Trim(),
    };
}

```

```

    try
    {
        CustomerFunc.AddCustomer(Program.Connection, newCustomer);
        MessageBox.Show($"Successfully Inserted an New Customer!", "Inserted",
        MessageBoxButtons.OK, MessageBoxIcon.Information);
        ListCustomer.Add($"{txtCid.Text}.{newCustomer.CustomerName}");
    }
    catch (Exception ex) { MessageBox.Show(ex.Message, "Submitting",
    MessageBoxButtons.OK, MessageBoxIcon.Error); }
    LoadingDataCustomer();
}
private void Select_Handling_Customer(object? sender, EventArgs e)
{
    if (LsCustomer.SelectedItem != null)
    {
        string? customerData = LsCustomer.SelectedItems[0].ToString();
        if (customerData == null) return;
        string[] tempData = customerData.Split('.');
        try
        {
            effectedCustomer = CustomerFunc.GetOneCustomer(Program.Connection,
            int.Parse(tempData[0].Trim()));
            txtCid.Text = effectedCustomer.CustomerID.ToString();
            txtCusName.Text = effectedCustomer.CustomerName;
            cbSex.Text = effectedCustomer.Gender;
            txtCusAge.Text = effectedCustomer.CusAge.ToString();
            txtCusInfo.Text = effectedCustomer.CustomerContactInfo;
        }
        catch (Exception ex)
        {
            MessageBox.Show($"{ex.Message}");
        }
    }
}
}

```

```

private void DoClickClear(object? sender, EventArgs e)
{
    txtCid.Text = (CustomerCount + 1).ToString();
    txtCusName.Clear();
    txtCusAge.Clear();
    cbSex.SelectedItem = null;
    txtCusInfo.Clear();
}
private void CustomerForm_Load(object sender, EventArgs e)
{
    LoadingDataCustomer();
    txtCid.Text = (CustomerCount + 1).ToString();
    foreach (string customer in LsCustomer.Items)
    {
        ListCustomer.Add(customer);
        txtSearch.CharacterCasing = CharacterCasing.Normal;
    }
}
private void LoadingDataCustomer()
{
    try
    {
        var result = CustomerFunc.GetAllCustomer(Program.Connection);
        if (result.LastOrDefault() != null) { CustomerCount =
result.LastOrDefault().CustomerID; }
        else { CustomerCount = 0; }
        LsCustomer.Items.Clear();
        foreach (var customer in result)
        {
            LsCustomer.Items.Add($"{customer.CustomerID}.
{customer.CustomerName}");
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message, "Retriving customer", MessageBoxButtons.OK,
MessageBoxIcon.Error);
    }
}
}

```

II. Model ទី 2 (DataGridView control)

Route's Form

© Copyright ISAD_M3_Group5(2024)



Route ID :

Origin :

Destination :

Distance :

Search ID:

Insert Update Clear Back Log Out

	RouteID	Origin	Destination	Distance
▶	3	pp	kandal	123km
	1001	pp	kpc	120km
	1002	pp	kps	100km
	1003	Phnom Pen	Kratie	120km

```

using PAGTMS.Models;
using PAGTMS.Utills;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.Entity;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Controls;
using System.Windows.Forms;
using CharacterCasing = System.Windows.Forms.CharacterCasing;

```

```
namespace PAGTMS;
```

```

public partial class RouteForm : Form
{
    Route? effectedRoute = null;
    List<DataGridViewRow> dataGridRoute = new List<DataGridViewRow>();
    int RouteCount = 0;
    private MainForm mainFormReference;
    int IndexOfUpdateRoute;

```

```

public RouteForm(MainForm mainForm)
{
    InitializeComponent();
    mainFormReference = mainForm;
    this.FormClosed += new FormClosedEventHandler( (_, _) => mainFormReference.Show());
    btnBack.Click += ( _, _) => this.Close();
    btnInsert.Click += DoClickInsertRoute;
    btnUpdate.Click += DoClickUpdateRoute;
    dgvRoute.Click += Select_Handling_Route;
    txtSearch.TextChanged += SearchChangedFunc;
    btnClear.Click += DoClickClear;
    dataGridRoute.Clear();
    btnLogOut.Click += ( _, _) => { this.Hide(); new LoginForm().Show(); };
}

private void SearchChangedFunc(object? sender, EventArgs e)
{
    if (string.IsNullOrEmpty(txtSearch.Text) == false)
    {
        dgvRoute.Rows.Clear();
        foreach (DataGridViewRow route in dataGridRoute)
        {
            if (route.Cells[0].Value.ToString().StartsWith(txtSearch.Text))
            {
                dgvRoute.Rows.Add(route);
            }
        }
    }
    else if (txtSearch.Text == "")
    {
        dgvRoute.Rows.Clear();
        foreach (DataGridViewRow route in dataGridRoute)
        {
            dgvRoute.Rows.Add(route);
        }
    }
}

private void DoClickClear(object? sender, EventArgs e)
{
    txtRid.Text = (RouteCount + 1).ToString();
    txtOrigin.Clear();
    txtDes.Clear();
    txtDis.Clear();
}

```

```

private void DoClickUpdateRoute(object? sender, EventArgs e)
{
    if (effecteatedRoute != null)
    {
        effecteatedRoute.RouteID = int.Parse(txtRid.Text);
        effecteatedRoute.Origin = txtOrigin.Text.Trim();
        effecteatedRoute.Destination = txtDes.Text.Trim();
        effecteatedRoute.Distance = txtDis.Text.Trim();

        try
        {
            var result = RouteFunc.UpdateRoute(Program.Connection, effecteatedRoute);
            if (result == true)
                MessageBox.Show($"Successfully Update!", "Updated",
                MessageBoxButtons.OK, MessageBoxIcon.Information);
            foreach (DataGridViewRow checkRoute in dataGridRoute)
            {
                if (int.Parse(checkRoute.Cells[0].Value.ToString()) ==
                effecteatedRoute.RouteID)
                {
                    IndexOfUpdateRoute = checkRoute.Index;
                    dataGridRoute.Remove(checkRoute);
                    break;
                }
            }
            DataGridViewRow tempDataGridViewRow = new DataGridViewRow();
            tempDataGridViewRow.CreateCells(dgvRoute);
            tempDataGridViewRow.Cells[0].Value = effecteatedRoute.RouteID;
            tempDataGridViewRow.Cells[1].Value = effecteatedRoute.Origin;
            tempDataGridViewRow.Cells[2].Value = effecteatedRoute.Destination;
            tempDataGridViewRow.Cells[3].Value = effecteatedRoute.Distance;
            dataGridRoute.Insert(IndexOfUpdateRoute, tempDataGridViewRow);
        }
        catch (Exception ex)
        {
            MessageBox.Show(ex.Message, "Update Failed ", MessageBoxButtons.OK,
            MessageBoxIcon.Error);
        }
    }
    LoadingDataRoute();
}

```

```

private void DoClickInsertRoute(object? sender, EventArgs e)
{
    if (int.TryParse(txtRid.Text, out int routeID) == false)
    {
        MessageBox.Show("Route Id is required", "Creating",
            MessageBoxButtons.OK, MessageBoxIcon.Stop);
        return;
    }
    if (txtOrigin.Text == "")
    {
        MessageBox.Show("Origin is required", "Creating",
            MessageBoxButtons.OK, MessageBoxIcon.Stop);
        return;
    }
    if (txtDes.Text == "")
    {
        MessageBox.Show("Destination is required", "Creating",
            MessageBoxButtons.OK, MessageBoxIcon.Stop);
        return;
    }
    if (txtDis.Text == "")
    {
        MessageBox.Show("Distance is required", "Creating",
            MessageBoxButtons.OK, MessageBoxIcon.Stop);
        return;
    }
    Route newRoute = new Route()
    {
        RouteID = routeID,
        Origin = txtOrigin.Text.Trim(),
        Destination = txtDes.Text.Trim(),
        Distance = txtDis.Text.Trim(),
    };
    try
    {
        RouteFunc.AddRoute(Program.Connection, newRoute);
        MessageBox.Show($"Successfully Insert!", "Inserted", MessageBoxButtons.OK,
            MessageBoxIcon.Information);
        DataGridViewRow tempDataGridViewRow = new DataGridViewRow();
        tempDataGridViewRow.CreateCells(dgvRoute);
        tempDataGridViewRow.Cells[0].Value = txtRid.Text;
        tempDataGridViewRow.Cells[1].Value = newRoute.Origin;
        tempDataGridViewRow.Cells[2].Value = newRoute.Destination;
        tempDataGridViewRow.Cells[3].Value = newRoute.Distance;
        dataGridRoute.Add(tempDataGridViewRow);
    }
    catch (Exception ex) { MessageBox.Show(ex.Message, "Submitting",
        MessageBoxButtons.OK, MessageBoxIcon.Error); }
    LoadingDataRoute();
}

private void Select_Handling_Route(object? sender, EventArgs e)
{
    if (dgvRoute.CurrentRow == null) return;
    int id = (int)dgvRoute.CurrentRow.Cells[0].Value;
    try
    {
        effectedRoute = RouteFunc.GetOneRoute(Program.Connection, id);
        txtRid.Text = effectedRoute.RouteID.ToString();
        txtOrigin.Text = effectedRoute.Origin;
        txtDes.Text = effectedRoute.Destination;
        txtDis.Text = effectedRoute.Distance;
    }
}

```

```

        catch (Exception ex)
        {
            MessageBox.Show($"Here: {ex.Message}");
        }
    }
    private void LoadingDataRoute()
    {
        try
        {
            var result = RouteFunc.GetAllRoute(Program.Connection);
            if (result.LastOrDefault() != null) { RouteCount =
result.LastOrDefault().RouteID; }
            else { RouteCount = 0; }
            dgvRoute.Rows.Clear();
            foreach (var route in result)
            {
                dgvRoute.Rows.Add(route.RouteID, route.Origin, route.Destination,
                route.Distance);
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show(ex.Message, "Retriving Route", MessageBoxButtons.OK,
            MessageBoxIcon.Error);
        }
    }

    private void RouteForm_Load(object sender, EventArgs e)
    {
        LoadingDataRoute();
        txtRid.Text = (RouteCount + 1).ToString();
        foreach (DataGridViewRow route in dgvRoute.Rows)
        {
            dataGridRoute.Add(route);
            txtSearch.CharacterCasing = CharacterCasing.Lower;
        }
    }
}

```

Database Programming

I. Database Script

```

create table tbCustomer(
    CusID int primary key not null,
    CusName varchar(20) not null,
    CusAge smallint not null,
    CusGender varchar(7) not null,
    CusContactInfo varchar(20) not null
)
go
create table tbStaff(
    StaffID int primary key not null,
    StaffNameKH nvarchar (30) Collate
SQL_Latin1_General_CP850_Bin not null,
    StaffNameEN varchar (30) not null,
    Gender varchar (7) not null,
    BirthDate date not null,
    StaffPosition varchar (50) not null,
    HouseNo varchar (50) not null,
    StreetNo varchar (50)not null,
    Sangkat varchar (50)not null,
    Khann varchar (50)not null,
    Province varchar(50)not null,
    ContactNumber varchar(50)not null,
    PersonalNumber varchar(50)not null,
    Salary money not null,
    HiredDate date not null,
    Photo varbinary(max),
    StopWork bit
)
go
create table tbRoute(
    RouteID int primary key not null,
    Origin varchar(50) not null,
    Destination varchar(50) not null,
    Distance varchar(50) not null
)
go
create table tbVehicle(
    VehicleID int primary key not null,
    VType varchar(50) not null,
    Capacity varchar(50) not null,
    Lincense_Plate varchar (50) not null,
    VStatus varchar(50)
)
go
create table tbBooking(
    BookingID int primary key not null,
    BookingDate date not null,
    TotalPaid money not null,
    CusID int, CusName varchar(20), CusContactInfo varchar(20),
    RouteID int, Destination varchar(50),
    VehicleID int, VType varchar(50), VStatus varchar(50),
    StaffID int, StaffNameKh nvarchar(50), StaffPosition varchar(100),
    constraint FK_CusID_tbBooking foreign key(CusID) references tbCustomer(CusID)
    on delete cascade on update cascade,
    constraint FK_RouteID_tbBooking foreign key(RouteID) references tbRoute(RouteID)
    on delete cascade on update cascade,

```

```

        constraint FK_VehicleID_tbBooking foreign key(VehicleID) references
        tbVehicle(VehicleID)
        on delete cascade on update cascade,
        constraint FK_StaffID_tbBooking foreign key(StaffID) references tbStaff(StaffID)
        on delete cascade on update cascade
    )
go
create table tbTicket(
    TicketID INT PRIMARY KEY not null,
    SeatNumber Varchar(25) not null,
    Price MONEY not null,
    TDate DATE,
    CusID INT,CusContactInfo varchar(20),
    StaffID INT, StaffPosition varchar(100),
    constraint FK_CusID_tbTicket foreign key(CusID) references tbCustomer(CusID)
    on delete cascade on update cascade,
    constraint FK_StaffID_tbTicket foreign key(StaffID) references tbStaff(StaffID)
    on delete cascade on update cascade,
)
go
create table tbTransport(
    TransportID int primary key not null,
    TransportDate date ,
    TStatus varchar(50) ,
    CusID int,CusContactInfo varchar(20),
    RouteID int, Destination varchar(50),
    StaffID int, StaffPosition varchar(100),
    constraint FK_CusID_tbTransport foreign key(CusID) references tbCustomer(CusID)
    on delete cascade on update cascade,
    constraint FK_Route_tbTransport foreign key(RouteID) references tbRoute(RouteID)
    on delete cascade on update cascade,
    constraint FK_StaffID_tbTransport foreign key(StaffID) references tbStaff(StaffID)
    on delete cascade on update cascade
)
go
create table tbItem(
    ItemID int primary key not null,
    Category varchar(50) ,
    IDescription varchar(50) ,
    IWeight varchar(10),
    CusID int,CusName varchar(50),CusContactInfo varchar(20),
    TransportID int, TransportDate date,
    constraint FK_CusID_tbItem foreign key(CusID) references tbCustomer(CusID)
    on delete cascade on update cascade,
    constraint FK_TransportID_tbItem foreign key(TransportID) references
    tbTransport(TransportID)
    on delete no action on update no action
)
go
create table tbPayment(
    PaymentID INT PRIMARY KEY IDENTITY(1,1),
    PayDate DATE,
    PaidAmount money ,
    CusID INT , CusName varchar(50), CusContactInfo varchar(20),
    StaffID INT, StaffNameKh nvarchar(50), StaffPosition varchar(50),
    constraint FK_CusID_tbPayment foreign key(CusID) references tbCustomer(CusID)
    on delete cascade on update cascade,
    constraint FK_StaffID_tbPayment foreign key(StaffID) references tbStaff(StaffID)
    on delete cascade on update cascade
)
go
create table tbInvoice(
    InvoiceID INT PRIMARY KEY not null,
    InvoiceDate DATE NOT NULL,
    TotalAmount money NOT NULL,

```

```

        PaidAmount money NOT NULL,
        CusID INT NOT NULL, CusName varchar(50), CusContactInfo varchar(20),
        StaffID INT NOT NULL, StaffNameKh nvarchar(50), StaffPosition varchar(50),
        constraint FK_CusID_tbInvoice foreign key(CusID) references tbCustomer(CusID) on
        delete cascade on update cascade,
        constraint FK_StaffID_tbInvoice foreign key(StaffID) references tbStaff(StaffID) on
        delete cascade on update cascade
    )

go
create table tbUser(
    UserID int primary key not null,
    UserName varchar(20),
    UserPassword varchar(20),
    StaffID INT NOT NULL, StaffNameKh nvarchar(50), StaffPosition varchar(100),
    constraint FK_StaffID_tbUser foreign key(StaffID) references tbStaff(StaffID)
    on delete cascade on update cascade
)

CREATE PROCEDURE spReadALLStaff
AS
BEGIN
    SELECT StaffID,StaffNameKH FROM tbStaff
END;
GO
-- Read One Staff
CREATE PROCEDURE spReadOneStaff ( @sid INT )
AS
BEGIN
    SELECT TOP 1 * FROM tbStaff WHERE StaffID = @sid
END;
GO

-- Insert Staff
CREATE PROCEDURE spInsertStaff ( @sid INT, @snKH NVARCHAR (100), @snEN VARCHAR (100),
@gen VARCHAR (6), @bd DATE, @pos VARCHAR (100), @hn VARCHAR (50), @sn varchar(50),
@sk varchar (50),@khn varchar (50),@pro varchar(50),
@cn VARCHAR (10), @pn VARCHAR (10) = NULL, @sa money ,@hd DATE, @ph VARBINARY (max) = NULL
,@spw bit)
AS
BEGIN
    INSERT INTO tbStaff VALUES ( @sid, @snKH, @snEN, @gen, @bd, @pos, @hn,
@sn,@sk,@khn,@pro,
@cn, @pn,@sa, @hd, @ph ,@spw)
END;
GO

-- Update Staff
CREATE PROCEDURE spUpdateStaff ( @sid INT, @snKH NVARCHAR (100), @snEN VARCHAR (100),
@gen VARCHAR (6), @bd DATE, @pos VARCHAR (100), @hn VARCHAR (50), @sn varchar(50),
@sk varchar (50),@khn varchar (50),@pro varchar(50),
@cn VARCHAR (10), @pn VARCHAR (10) = NULL,@sa money ,@hd DATE, @ph VARBINARY (max) = NULL
,@spw bit )
AS
BEGIN
    UPDATE tbStaff SET
        StaffID = @sid,
        StaffNameKH = @snKH,
        StaffNameEN = @snEN,
        Gender = @gen,
        BirthDate = @bd,
        StaffPosition = @pos,
        HouseNo= @hn,
        StreetNo = @sn,

```

```

        Sangkat = @sk,
        Khann = @khn,
        Province = @pro,
        ContactNumber = @cn,
        PersonalNumber = @pn,
        Salary = @sa,
        HiredDate = @hd,
        Photo = @ph,
        StopWork = @spw
        WHERE StaffID = @sid
END;
go
--Read all Customer
create proc spReadAllCustomer
as
Begin
Select CusID,CusName from tbCustomer
end;
go
--Read one Customer
create proc spReadOneCustomer(@cid int)
as
Begin
select top 1* From tbCustomer where CusID = @cid
end;
go
--Insert Customer
create proc spInsertCustomer(@cid int,@cn varchar(50),@ca smallint,@cg varchar(7),
@cci varchar(20))
as
begin
insert into tbCustomer values(@cid,@cn,@ca,@cg,@cci)
end;
go
--Update Customer
create proc spUpdateCustomer(@cid int,@cn varchar(50),@ca smallint,@cg varchar(7),
@cci varchar(20))
as
begin
        update tbCustomer set
        CusID = @cid,
        CusName =@cn,
        CusAge=@ca,
        CusGender=@cg,CusContactInfo = @cci
        where CusID =@cid
end;
go

--Read all Vehicle
create proc spReadAllVehicle
as
Begin
Select * from tbVehicle
end;
go
--Read one Vehicle
create proc spReadOneVehicle(@vid int)
as
Begin
select top 1* From tbVehicle where VehicleID = @vid
end;
go
--Insert Vehicle
create proc spInsertVehicle(@vid int,@vt varchar(50),@cp varchar(50),
@lp varchar(50),@vs varchar(50))

```

```

as
begin
insert into tbVehicle values(@vid,@vt,@cp,@lp,@vs)
end;
go
--Update Vehicle
create proc spUpdateVehicle(@vid int,@vt varchar(50),@cp varchar(50),
@lp varchar(50),@vs varchar(50))
as
begin
    update tbVehicle set
        VehicleID = @vid,
        VType = @vt,
        Capacity = @cp,
        Lincense_Plate =@lp,
        VStatus= @vt
    where VehicleID =@vid
end;
go

--Read all Route
create proc spReadAllRoute
as
Begin
Select * from tbRoute
end;
go
--Read one Route
create proc spReadOneRoute(@rid int)
as
Begin
select top 1* From tbRoute where RouteID = @rid
end;
go
--Insert Route
create proc spInsertRoute(@rid int,@ori varchar(50),@des varchar(50),
@dis varchar(50))
as
begin
insert into tbRoute values(@rid,@ori,@des,@dis)
end;
go
--Update Route
create proc spUpdateRoute(@rid int,@ori varchar(50),@des varchar(50),
@dis varchar(50))
as
begin
    update tbRoute set
        RouteID = @rid,
        Origin= @ori,
        Destination = @des,
        Distance =@dis
    where RouteID =@rid
end;
go
--Read all Booking
CREATE PROCEDURE spReadALLBooking
AS
BEGIN
    SELECT * FROM tbBooking
END;
go
-- Read One Booking
CREATE PROCEDURE spReadOneBooking ( @bookingid INT )
AS

```

```

BEGIN
    SELECT TOP 1 * FROM tbBooking WHERE BookingID = @bookingid
END;
go
-- Insert booking
CREATE PROCEDURE spInsertBooking ( @bookingid int,@bookingdate date,@totalpaid money,
@cusid int,@cusname varchar(50),@cusinfo varchar(20),
@routeid int, @des varchar(20),
@vehicleid int, @vehicletype varchar(50),@vehiclestatus varchar(50),
@stid INT, @stkh NVARCHAR (50), @stpos VARCHAR (100))
AS
BEGIN
    INSERT INTO tbBooking VALUES (@bookingid,@bookingdate,@totalpaid,
@cusid,@cusname,@cusinfo,
@routeid,@des,
@vehicleid,@vehicletype,@vehiclestatus,
@stid,@stkh,@stpos);

END;
go
-- Update Booking
Create PROCEDURE spUpdateBooking ( @bookingid int,@bookingdate date,@totalpaid money,
@cusid int,@cusname varchar(50),@cusinfo varchar(20),
@routeid int, @des varchar(20),
@vehicleid int, @vehicletype varchar(50),@vehiclestatus varchar(50),
@stid INT, @stkh NVARCHAR (50), @stpos VARCHAR (100))
AS
BEGIN
    UPDATE tbBooking SET
        BookingID= @bookingid,
        BookingDate = @bookingdate,
        TotalPaid = @totalpaid,
        CusID = @cusid,
        CusName = @cusname,
        CusContactInfo =@cusinfo,
        RouteID = @routeid,
        Destination=@des,
        VehicleID =@vehicleid,
        VType = @vehicletype,
        VStatus = @vehiclestatus,
        StaffID = @stid,
        StaffNameKh = @stkh,
        StaffPosition = @stpos
        WHERE BookingID = @bookingid

END;
GO
--Read all Ticket
CREATE PROCEDURE spReadALLTicket
AS
BEGIN
    SELECT * FROM tbTicket
END;
go
-- Read One Ticket
CREATE PROCEDURE spReadOneTicket ( @ticketid INT )
AS
BEGIN
    SELECT TOP 1 * FROM tbTicket WHERE TicketID = @ticketid
END;
go
-- Insert Ticket
CREATE PROCEDURE spInsertTicket (@ticketid int,@seatnum varchar(25),
@price money,@ticketdate date,
@cusid int,@cusinfo varchar(20),
@stid INT,@stpos VARCHAR (100))

```

```

AS
BEGIN
    INSERT INTO tbTicket VALUES ( @ticketid,@seatnum,@price,@ticketdate,
    @cusid,@cusinfo,
    @stid,@stpos)

END;
go
-- Update Ticket
Create PROCEDURE spUpdateTicket (@ticketid int,@seatnum varchar(50),
@price money,@ticketdate date,
@cusid int,@cusinfo varchar(20),
@stid INT,@stpos VARCHAR (100))
AS
BEGIN
    UPDATE tbTicket SET
        TicketID= @ticketid,
        SeatNumber =@seatnum,
        Price =@price,
        TDate = @ticketdate,
        CusID = @cusid,
        CusContactInfo =@cusinfo,
        StaffID = @stid,
        StaffPosition = @stpos
        WHERE TicketID = @ticketid

END;
GO
--Read all Transport
CREATE PROCEDURE spReadALLTransport
AS
BEGIN
    SELECT * FROM tbTransport
END;
go
--Read one Transport
CREATE PROCEDURE spReadOneTransport(@tranid INT)
AS
BEGIN
    SELECT TOP 1 * FROM tbTransport WHERE TransportID = @tranid
END;
go
--Insert Transport
CREATE PROCEDURE spInsertTransport(@tranid INT,@trandate date,@transtatus varchar(50),
@cusid INT,@cusinfo varchar(20),@routeid int,@rdes varchar(50),
@stid int,@stpos varchar(20))
AS
BEGIN
    INSERT INTO tbTransport
VALUES(@tranid,@trandate,@transtatus,@cusid,@cusinfo,@routeid,@rdes,@stid,@stpos)
END;
go
--Update Transport
CREATE PROCEDURE spUpdateTransport(@tranid INT,@trandate date,@transtatus varchar(50),
@cusid INT,@cusinfo varchar(20),@routeid int,@rdes varchar(50),
@stid int,@stpos varchar(20))
AS
BEGIN
    UPDATE tbTransport SET
        TransportID = @tranid,
        TransportDate = @trandate,
        TStatus = @transtatus,
        CusID = @cusid,
        CusContactInfo = @cusinfo,
        RouteID = @routeid,
        Destination = @rdes,

```

```

        StaffID = @stid,
        StaffPosition = @stpos
    WHERE TransportID = @trandid
END;
go
-- insert Item
CREATE PROCEDURE spInsertItem(@itid INT ,@cate VARCHAR(50),@des VARCHAR(50) ,@wei
varchar(10),@cusid INT ,@cusna varchar(50),@cusco varchar(20),@tid INT ,@td varchar(50))
AS
BEGIN
    INSERT INTO tbItem VALUES(@itid,@cate,@des,@wei,@cusid,@cusna,@cusco,@tid,@td)
END;
go
-- Read all Item
CREATE PROCEDURE spReadALLItem
AS
BEGIN
    SELECT * FROM tbItem
END;
go
--Read one Item
CREATE PROCEDURE spReadOneItem(@itid INT)
AS
BEGIN
    SELECT TOP 1 * FROM tbItem WHERE ItemID = @itid
END;
go
--update Item
CREATE PROCEDURE spUpdateItem(@itid INT ,@cate VARCHAR(50),@des VARCHAR(50) ,@wei
varchar(10),@cusid INT ,@cusna varchar(50),@cusco varchar(20),@tid INT ,@td varchar(50))
AS
BEGIN
    UPDATE tbItem SET
        ItemID = @itid,
        Category = @cate,
        IDescription = @des,
        IWeight = @wei,
        CusID = @cusid,
        CusName =@cusna,
        CusContactInfo = @cusco,
        TransportID = @tid,
        TransportDate = @td
    WHERE ItemID = @itid
END;
GO
-- Read all Payment
CREATE PROCEDURE spReadALLPayment
AS
BEGIN
    SELECT PaymentID,CusName FROM tbPayment
END;
GO
-- Read One Payment
CREATE PROCEDURE spReadOnePayment ( @pid INT )
AS
BEGIN
    SELECT TOP 1 * FROM tbPayment WHERE PaymentID = @pid
END;
GO
-- Insert Payment
CREATE PROCEDURE spInsertPayment ( @pid int output ,@pdate date,@paidamount money,
@cusid int,@cusname varchar(50),@cusinfo varchar(20),
@stid INT, @stkh NVARCHAR (50), @stpos VARCHAR (100))
AS
BEGIN

```

```

INSERT INTO tbPayment VALUES
(@pdate,@paidamount,@cusid,@cusname,@cusinfo,@stid,@stkh,@stpos);
SET @pid = SCOPE_IDENTITY();

END;
GO
-- Update Payment
Create PROCEDURE spUpdatePayment ( @pid int,@pdate date,@paidamount money,
@cusid int,@cusname varchar(50),@cusinfo varchar(20),
@stid INT, @stkh NVARCHAR (50), @stpos VARCHAR (100))
AS
BEGIN
    UPDATE tbPayment SET
        PayDate =@pdate,
        PaidAmount = @paidamount,
        CusID = @cusid,
        CusName = @cusname,
        CusContactInfo =@cusinfo,
        StaffID = @stid,
        StaffNameKh = @stkh,
        StaffPosition = @stpos
        WHERE PaymentID = @pid
END;
GO
--Read all Invoice
CREATE PROCEDURE spReadALLInvoice
AS
BEGIN
    SELECT InvoiceID,CusName FROM tbInvoice
END;
GO
-- Read One Invoice
CREATE PROCEDURE spReadOneInvoice ( @iid INT )
AS
BEGIN
    SELECT TOP 1 * FROM tbInvoice WHERE InvoiceID = @iid
END;
GO
-- Insert Invoice
CREATE PROCEDURE spInsertInvoice ( @iid int,@idate date,@totalamount money,@paidamount money,
@cusid int,@cusname varchar(50),@cusinfo varchar(20),
@stid INT, @stkh NVARCHAR (50), @stpos VARCHAR (100))
AS
BEGIN
    INSERT INTO tbInvoice VALUES
    (@iid,@idate,@totalamount,@paidamount,@cusid,@cusname,@cusinfo,@stid,@stkh,@stpos)
END;
GO
-- Update Invoice
CREATE PROCEDURE spUpdateInvoice ( @iid int,@idate date,@totalamount money,@paidamount money,

```

```

@cusid int,@cusname varchar(50),@cusinfo varchar(20),
@stid INT, @stkh NVARCHAR (50), @stpos VARCHAR (100))
AS
BEGIN

    UPDATE tbInvoice SET
        InvoiceID =@iid,
        InvoiceDate =@idate,
        TotalAmount = @totalamount,
        PaidAmount = @paidamount,
        CusID = @cusid,
        CusName = @cusname,
        CusContactInfo =@cusinfo,
        StaffID = @stid,
        StaffNameKh = @stkh,
        StaffPosition = @stpos
        WHERE InvoiceID = @iid

END;
go
--Read all User
CREATE PROCEDURE spReadALLUser
AS
BEGIN
    SELECT UserID,UserName FROM tbUser
END;
GO
-- Read One User
CREATE PROCEDURE spReadOneUser ( @uid INT )
AS
BEGIN
    SELECT TOP 1 * FROM tbUser WHERE UserID = @uid
END;
GO

-- Insert User
CREATE PROCEDURE spInsertUser ( @uid int,@uname varchar(20),@upass varchar(20) ,@stid INT,
@stkh NVARCHAR (100), @stpos VARCHAR (100))
AS
BEGIN
    INSERT INTO tbUser VALUES (@uid,@uname,@upass,@stid,@stkh,@stpos)
END;
GO

-- Update User
CREATE PROCEDURE spUpdateUser ( @uid int,@uname varchar(20),@upass varchar(20) ,@stid INT,
@stkh NVARCHAR (100), @stpos VARCHAR (100))
AS
BEGIN
    UPDATE tbUser SET
        UserID = @uid,
        UserName = @uname,
        UserPassword = @upass,
        StaffID = @stid,
        StaffNameKh = @stkh,
        StaffPosition = @stpos
        WHERE UserID = @uid
END;

```

```

CREATE TRIGGER trgUpdateStaffNameAndPosition_tbUser
ON tbStaff
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE tbUser
    SET tbUser.StaffNameKh = i.StaffNameKH,
        tbUser.StaffPosition = i.StaffPosition
FROM tbUser u
    INNER JOIN inserted i ON u.StaffID = i.StaffID
    INNER JOIN deleted d ON u.StaffID = d.StaffID
    WHERE i.StaffNameKH <> d.StaffNameKH OR
        i.StaffPosition <> d.StaffPosition;
END;
go
CREATE TRIGGER trgUpdateStaffNameAndPosition_tbBooking
ON tbStaff
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE tbBooking
    SET tbBooking.StaffNameKh = i.StaffNameKH,
        tbBooking.StaffPosition = i.StaffPosition
FROM tbBooking b
    INNER JOIN inserted i ON b.StaffID = i.StaffID
    INNER JOIN deleted d ON b.StaffID = d.StaffID
    WHERE i.StaffNameKH <> d.StaffNameKH OR
        i.StaffPosition <> d.StaffPosition;
END;
go
CREATE TRIGGER trgUpdateStaffNameAndPosition_tbInvoice
ON tbStaff
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE tbInvoice
    SET tbInvoice.StaffNameKh = i.StaffNameKH,
        tbInvoice.StaffPosition = i.StaffPosition
FROM tbInvoice ii
    INNER JOIN inserted i ON ii.StaffID = i.StaffID
    INNER JOIN deleted d ON ii.StaffID = d.StaffID
    WHERE i.StaffNameKH <> d.StaffNameKH OR
        i.StaffPosition <> d.StaffPosition;
END;
go
CREATE TRIGGER trgUpdateStaffNameAndPosition_tbPayment
ON tbStaff
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE tbPayment
    SET tbPayment.StaffNameKh = i.StaffNameKH,
        tbPayment.StaffPosition = i.StaffPosition
FROM tbPayment p
    INNER JOIN inserted i ON p.StaffID = i.StaffID
    INNER JOIN deleted d ON p.StaffID = d.StaffID

```

```

WHERE i.StaffNameKH <> d.StaffNameKH OR
      i.StaffPosition <> d.StaffPosition;
END;
go
CREATE TRIGGER trgUpdateStaffNameAndPosition_tbTicket
ON tbStaff
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE tbTicket
    SET tbTicket.StaffPosition = i.StaffPosition
    FROM tbTicket ti
    INNER JOIN inserted i ON ti.StaffID = i.StaffID
    INNER JOIN deleted d ON ti.StaffID = d.StaffID
    WHERE i.StaffPosition <> d.StaffPosition;
END;
go
CREATE TRIGGER trgUpdateStaffNameAndPosition_tbTransport
ON tbStaff
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE tbTransport
    SET tbTransport.StaffPosition = i.StaffPosition
    FROM tbTransport tr
    INNER JOIN inserted i ON tr.StaffID = i.StaffID
    INNER JOIN deleted d ON tr.StaffID = d.StaffID
    WHERE i.StaffPosition <> d.StaffPosition;
END;
go
CREATE TRIGGER trgUpdateCusNameandContactInfo_tbBooking
ON tbCustomer
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE tbBooking
    SET tbBooking.CusName = i.CusName,
        tbBooking.CusContactInfo = i.CusContactInfo
    FROM tbBooking bk
    INNER JOIN inserted i ON bk.CusID = i.CusID
    INNER JOIN deleted d ON bk.CusID = d.CusID
    WHERE i.CusName <> d.CusName or
        i.CusContactInfo<>d.CusContactInfo;
END;
go
CREATE TRIGGER trgUpdateCusNameandContactInfo_tbInvoice
ON tbCustomer
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE tbInvoice
    SET tbInvoice.CusName = i.CusName,
        tbInvoice.CusContactInfo = i.CusContactInfo
    FROM tbInvoice bk
    INNER JOIN inserted i ON bk.CusID = i.CusID
    INNER JOIN deleted d ON bk.CusID = d.CusID
    WHERE i.CusName <> d.CusName or
        i.CusContactInfo<>d.CusContactInfo;

```

```

END;
go
CREATE TRIGGER trgUpdateCusNameandContactInfo_tbItem
ON tbCustomer
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE tbItem
    SET tbItem.CusName = i.CusName,
        tbItem.CusContactInfo = i.CusContactInfo
    FROM tbItem bk
    INNER JOIN inserted i ON bk.CusID = i.CusID

    INNER JOIN deleted d ON bk.CusID = d.CusID
    WHERE i.CusName <> d.CusName or
        i.CusContactInfo<>d.CusContactInfo;
END;
go
CREATE TRIGGER trgUpdateCusNameandContactInfo_tbPayment
ON tbCustomer
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE tbPayment
    SET tbPayment.CusName = i.CusName,
        tbPayment.CusContactInfo = i.CusContactInfo
    FROM tbPayment bk
    INNER JOIN inserted i ON bk.CusID = i.CusID
    INNER JOIN deleted d ON bk.CusID = d.CusID
    WHERE i.CusName <> d.CusName or
        i.CusContactInfo<>d.CusContactInfo;
END;
go
CREATE TRIGGER trgUpdateCusNameandContactInfo_tbTicket
ON tbCustomer
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE tbTicket
    SET tbTicket.CusContactInfo = i.CusContactInfo
    FROM tbTicket bk
    INNER JOIN inserted i ON bk.CusID = i.CusID
    INNER JOIN deleted d ON bk.CusID = d.CusID
    WHERE i.CusContactInfo<>d.CusContactInfo;
END;
go
CREATE TRIGGER trgUpdateCusNameandContactInfo_tbTransport
ON tbCustomer
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE tbTransport
    SET tbTransport.CusContactInfo = i.CusContactInfo
    FROM tbTransport bk
    INNER JOIN inserted i ON bk.CusID = i.CusID
    INNER JOIN deleted d ON bk.CusID = d.CusID
    WHERE i.CusContactInfo<>d.CusContactInfo;

```

```
END;
go
CREATE TRIGGER trgUpdateDestination_tbBooking
ON tbRoute
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE tbBooking
    SET tbBooking.Destination = i.Destination
    FROM tbBooking bk
    INNER JOIN inserted i ON bk.RouteID = i.RouteID
    INNER JOIN deleted d ON bk.RouteID = d.RouteID
    WHERE i.Destination<>d.Destination;
END;
go
CREATE TRIGGER trgUpdateDestination_tbTransport
ON tbRoute
```

